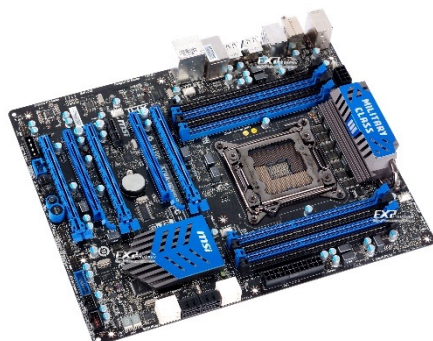




《计算机科学导论》课程

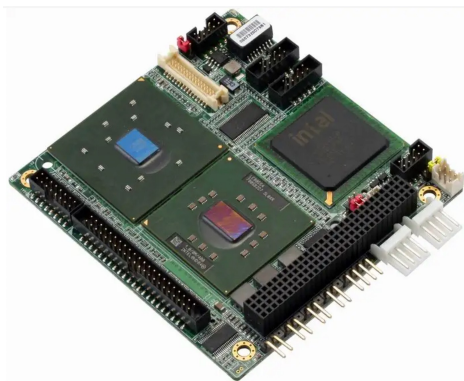
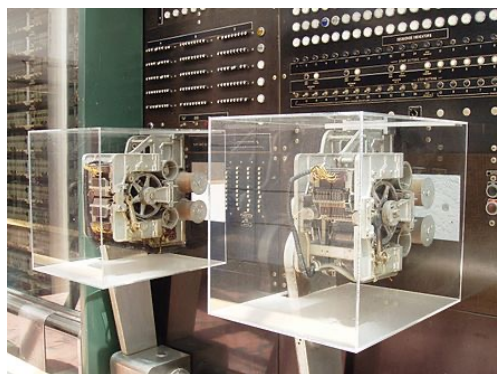
计算机系统结构

上海交通大学计算机科学导论课程组



致谢：本课件初稿由卢昊桢@CS2019M.SJTU编写，刘雨桐@CS2017D.SJTU、向嘉@CS2021M.SJTU修缮排版，孔令和、陈国兴、陈全添加内容；二稿由邓倩妮指导，高晓沅重新组织编写，洪义、高晓沅终校修订。

什么是计算机系统结构？



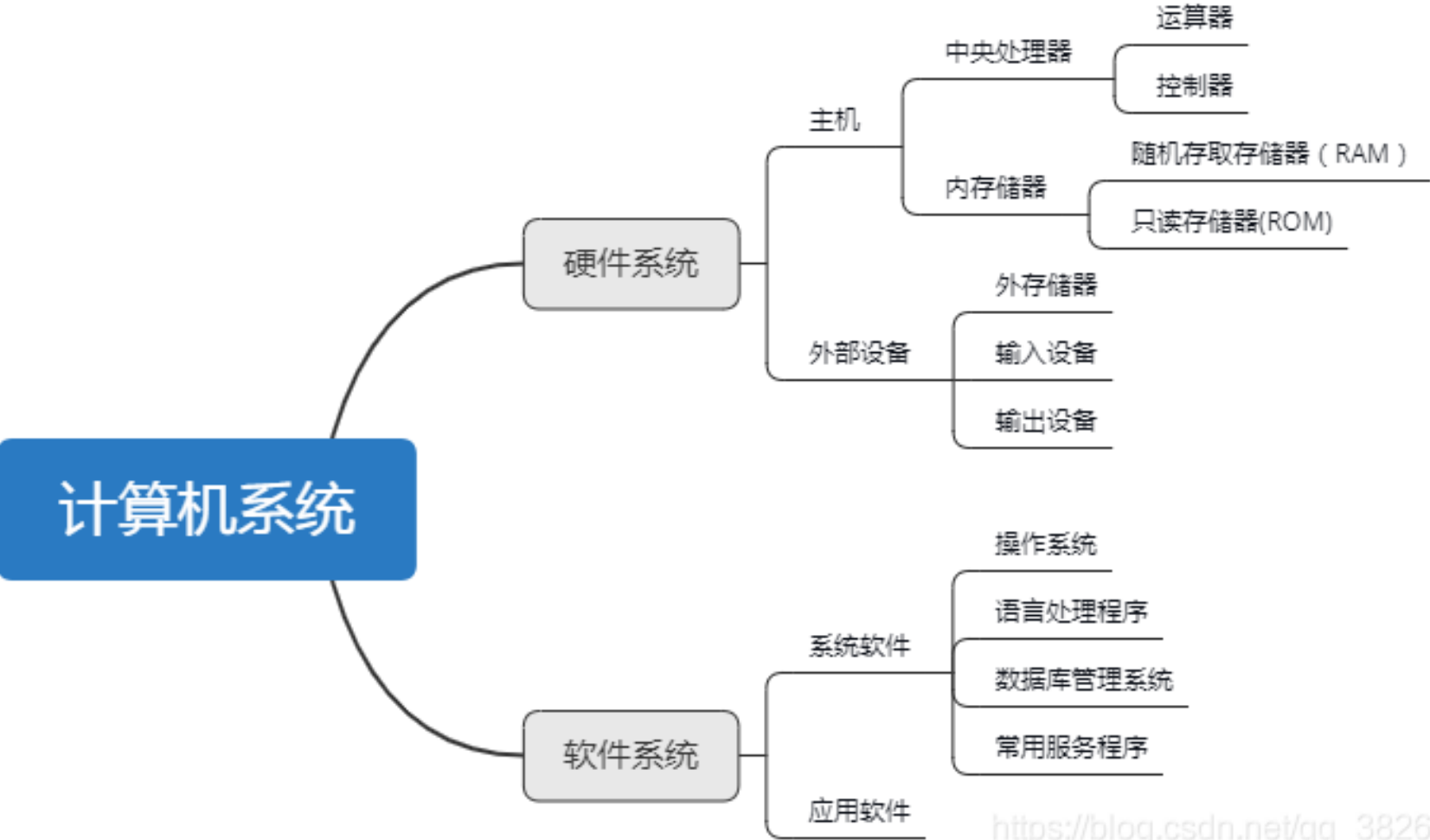
什么是计算机系统结构？

- Computer architecture is a set of rules and methods that describe the functionality, organization, and implementation of computer systems. -Wikipedia

如何设计计算机，使同样的芯片面积能发挥出更高的算力？

上层软件对高算力的需求促使计算机系统结构快速发展！

什么是计算机系统结构？



ZOL 模拟攒机

编辑推荐 硬件报价 DIY

模拟攒机 攒机指南 我的方案 网友方案 大师方案 配置单

本周热门: 1234 (来自: zygyf9)

请选择配件 * 号为必选项

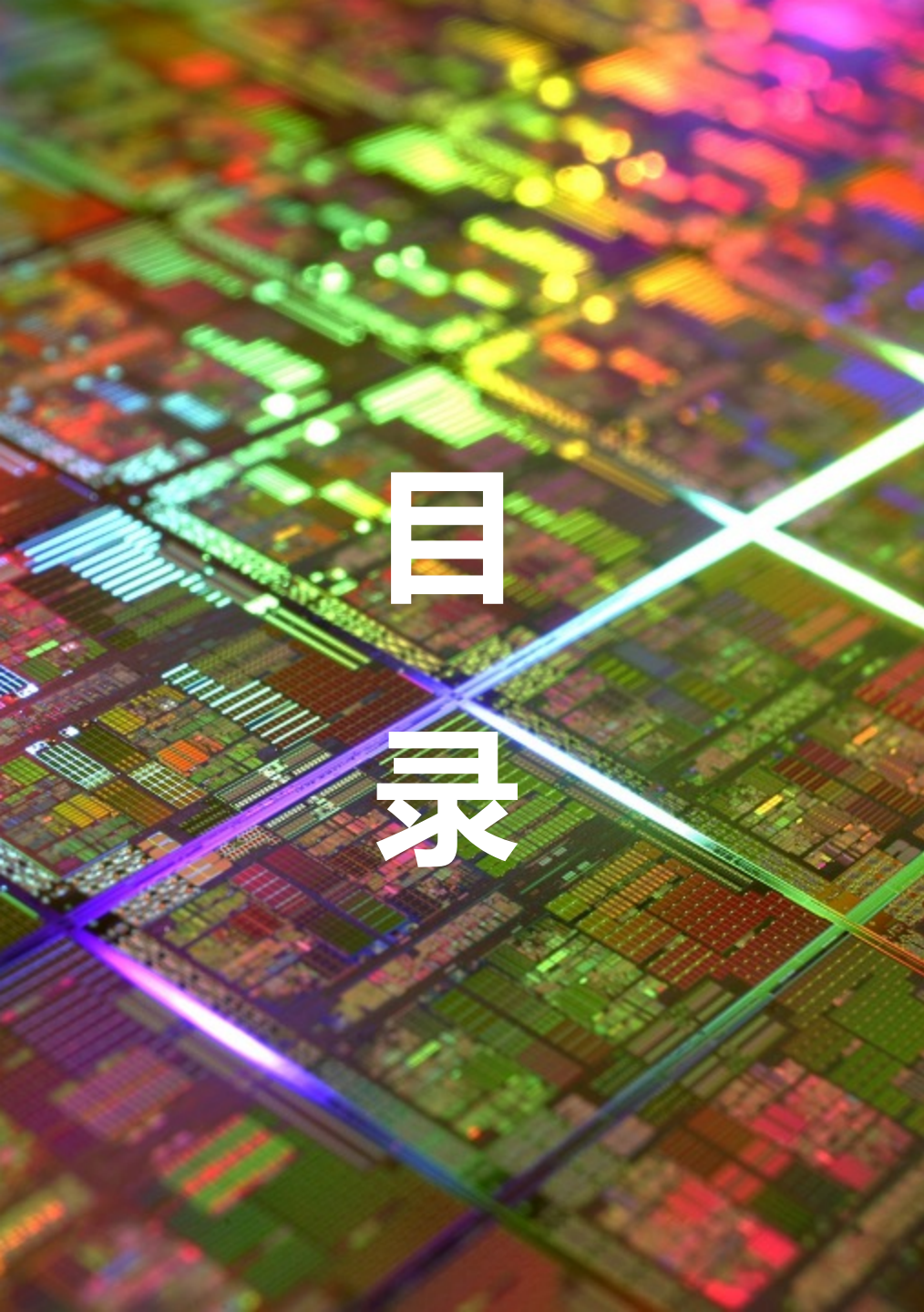
CPU * 主板 * 内存 * 硬盘 固态硬盘 显

装机配置单

您还未登录, 登录后才能预览和发表配置. 登录

CPU *		请选择商品	添加 x
主板 *		请选择商品	添加
内存 *		请选择商品	添加
硬盘		请选择商品	添加
固态硬盘		请选择商品	添加
显卡		请选择商品	添加
显示器		请选择商品	添加
机箱		请选择商品	添加
电源		请选择商品	添加

https://blog.csdn.net/qq_38263696



目 录

01

基础知识

02

CPU指令集

03

存储设备

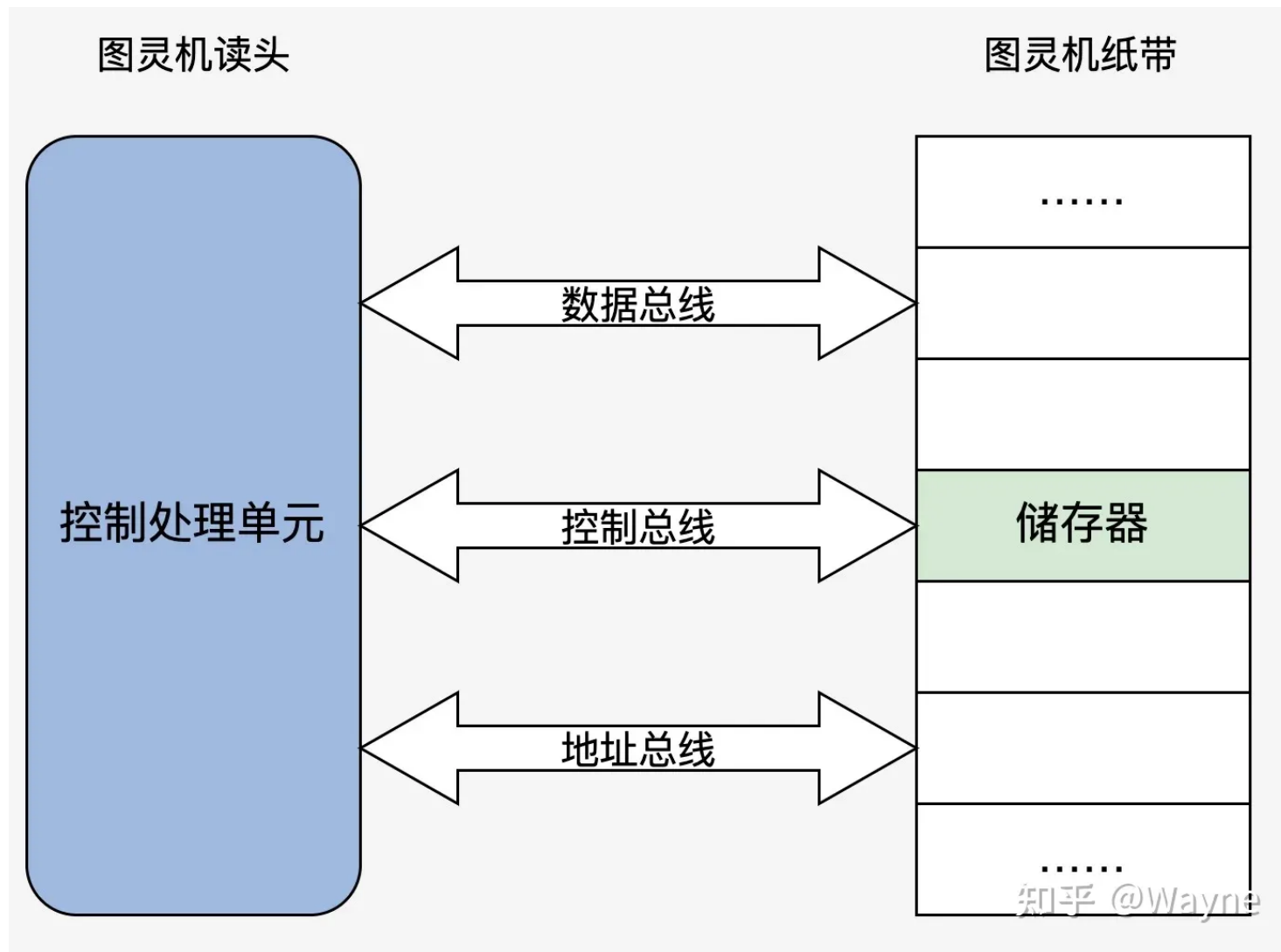
04

操作系统

01.基础知识：图灵机的工程实现

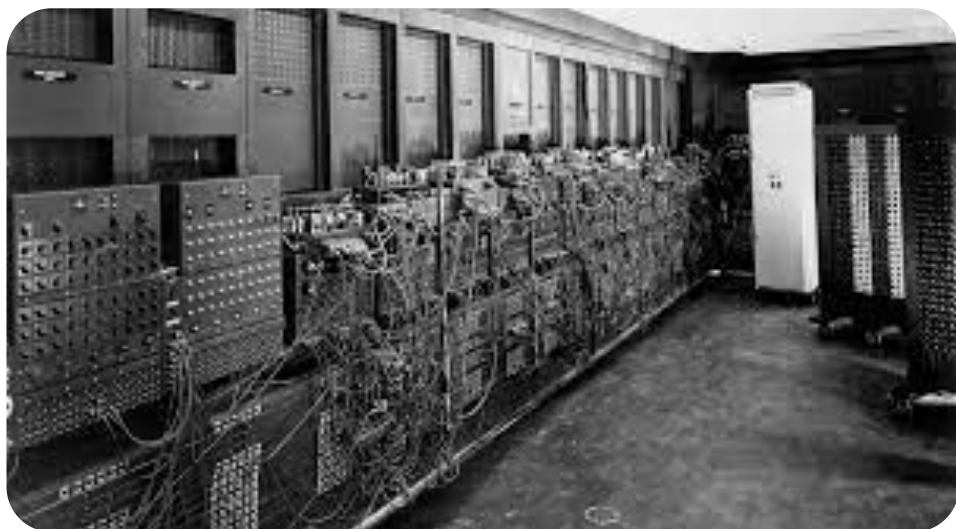
实现一个图灵机：

- 把程序和数据装入计算机；
- 能长期记住程序、数据的中间结果及最终运算结果；
- 完成各种算术、逻辑运算和数据传送等数据加工处理；
- 根据需要控制程序走向，并能根据指令控制机器的各部件协调操作；
- 能够按照要求将处理的数据结果显示给用户。

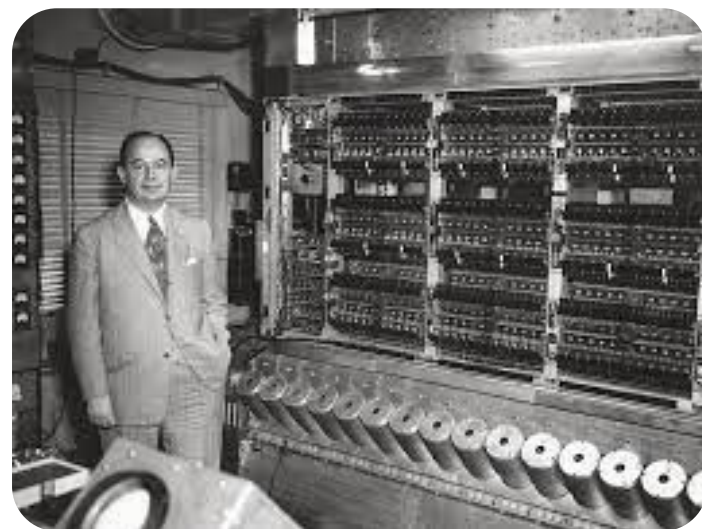


01.基础知识：EDVAC

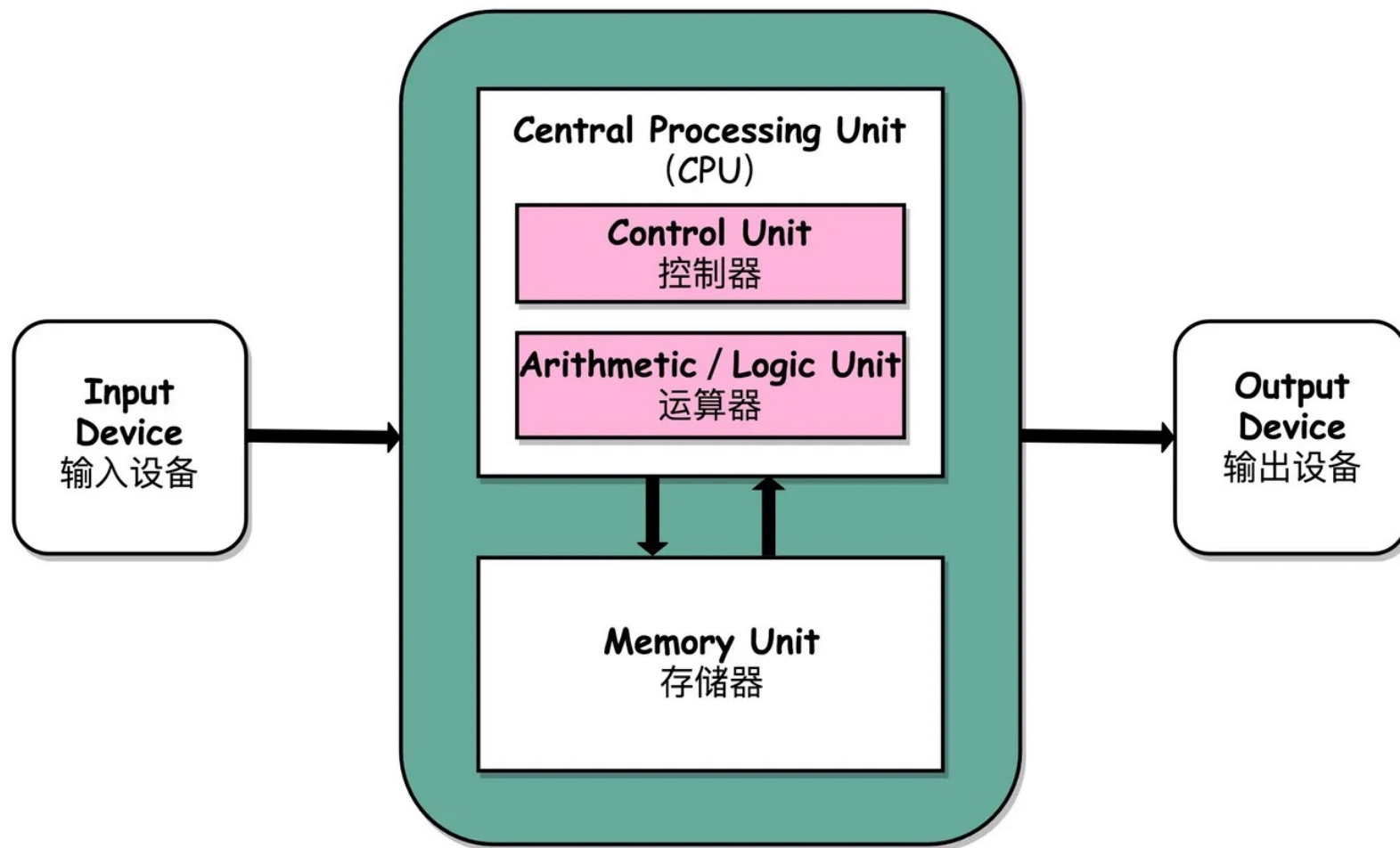
ENIAC: (Electronic Numerical Integrator And Calculator, 电子数字积分计算机) 世界上第一台计算机, 1946年在美国宾夕法尼亚大学投入运行, 采用十进制进行数据存储。奠定电子计算机的发展基础, 开辟了信息时代新纪元, 是人类第三次产业革命开始的标志。



EDVAC: (Electronic Discrete Variable Automatic Computer, 电子离散变量自动计算机)。世界上第一台采用二进制的计算机, 由**控制器、运算器、存储器、输入和输出设备**5部分组成。1945年3月由冯·诺伊曼与莫奇利、埃克特等提出, 1951年最终完成。



01.基础知识：计算机系统结构1—冯·诺依曼结构



冯·诺依曼结构

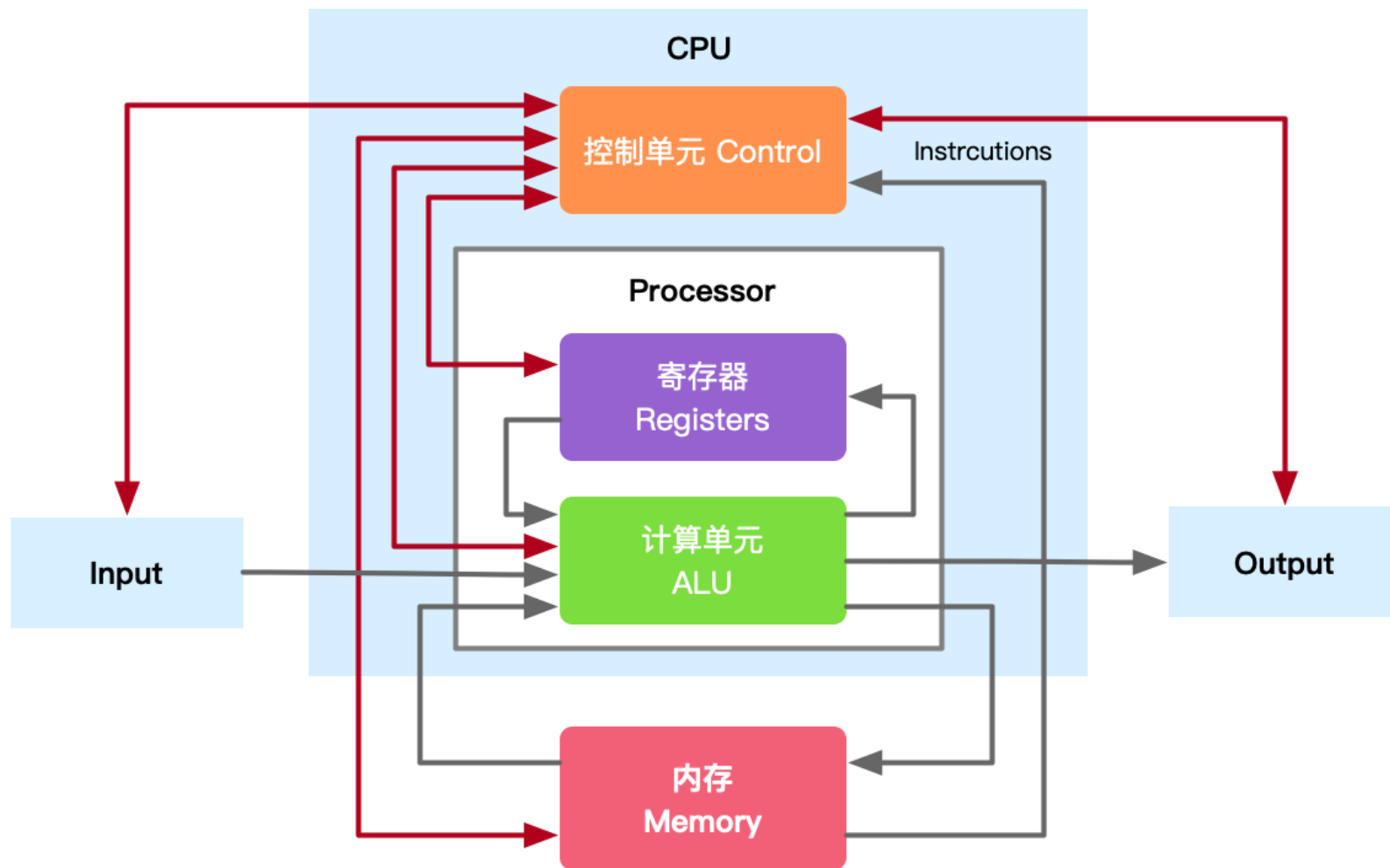


约翰·冯·诺依曼
John von Neumann
1903-1957

普林斯顿大学

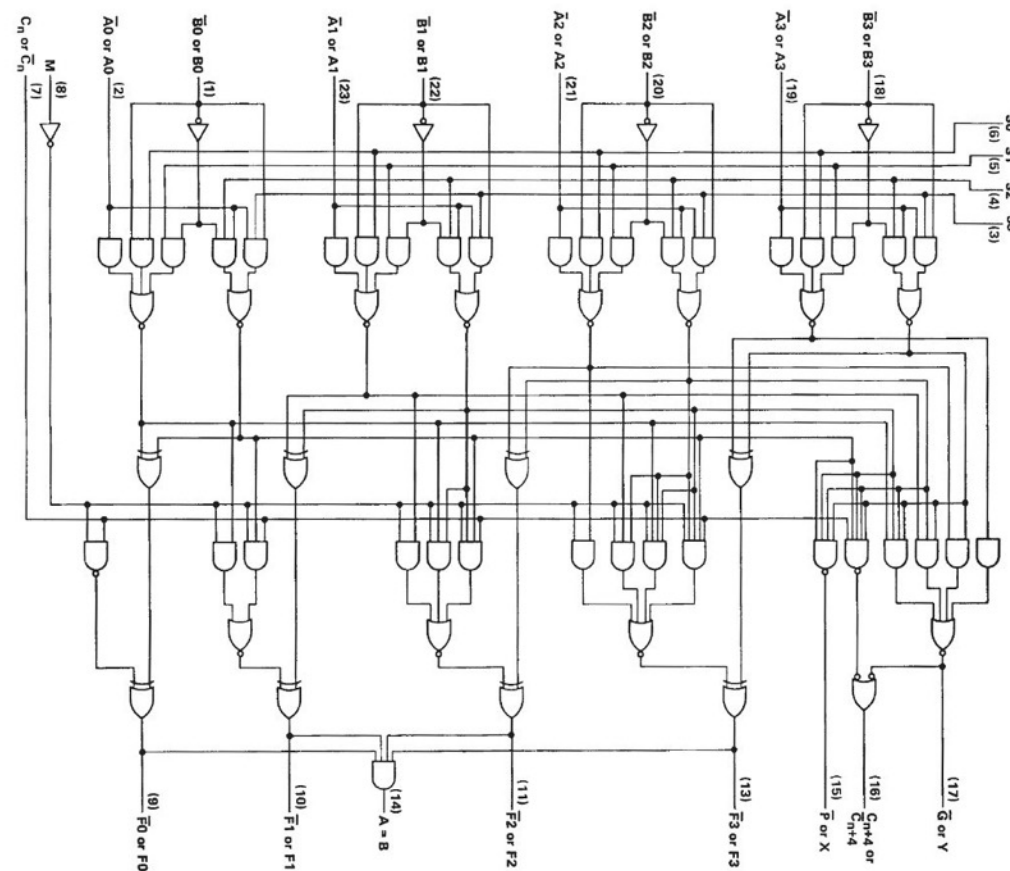
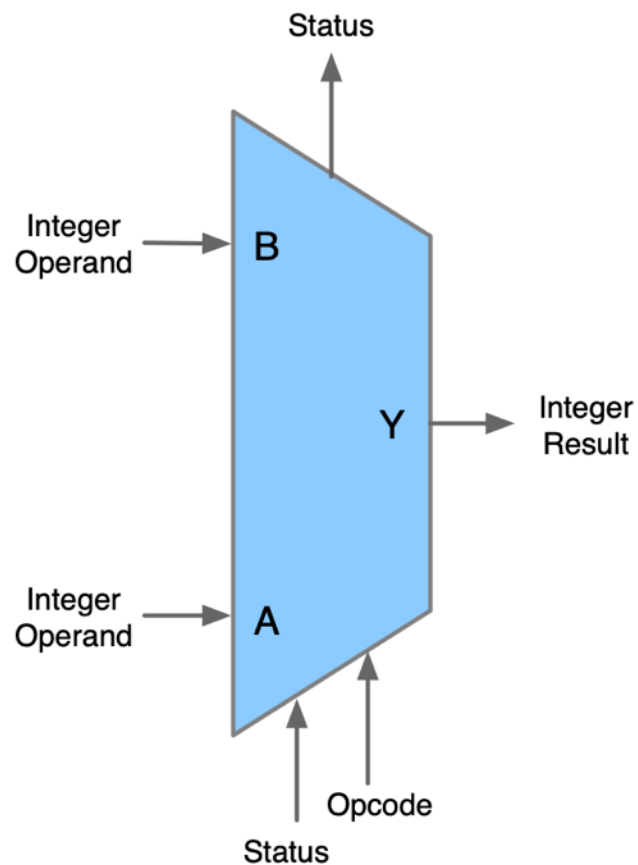
01.基础知识：中央处理单元（Central Processing Unit）

从彼时至今，无论CPU的具体实现怎么变、晶体管数量翻多少番，它的构成始终由控制器、运算器和寄存器这三大组成部分组成。



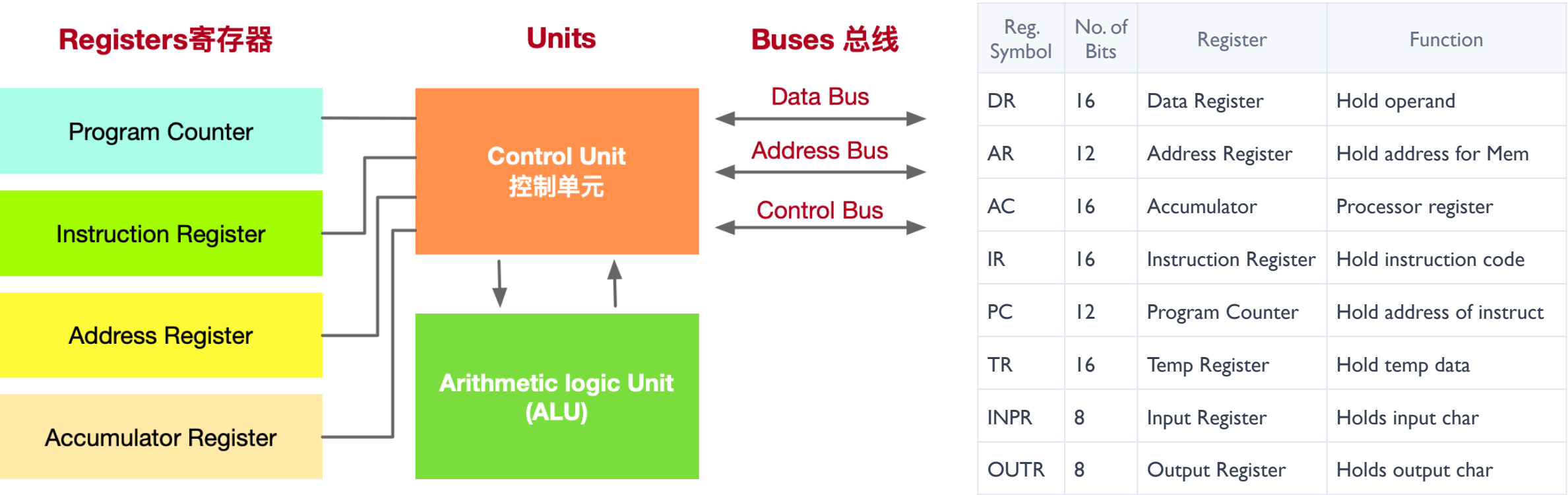
01.基础知识：CPU主要组成

运算器：也叫算术逻辑单元（ALU），负责算术运算和逻辑运算。



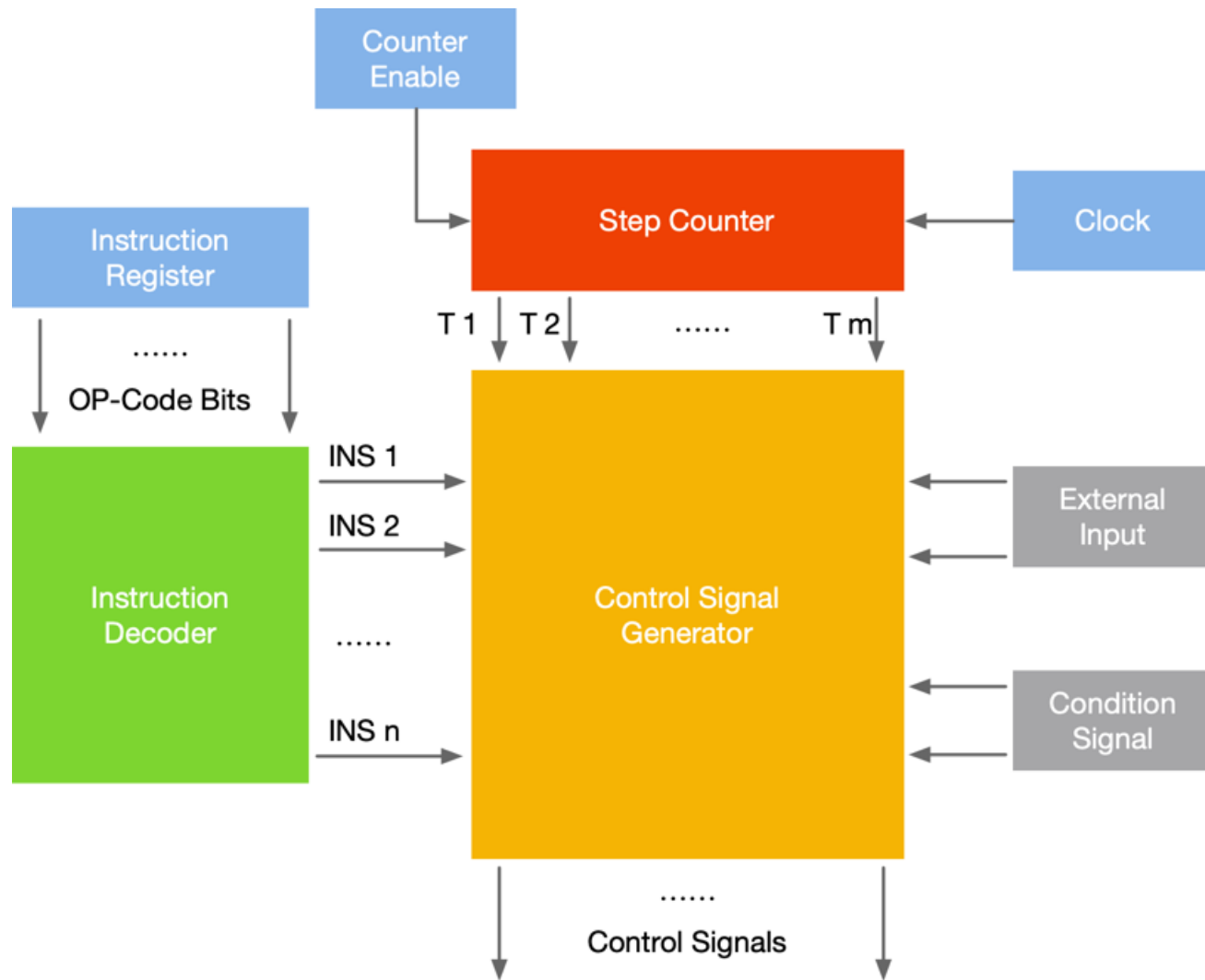
01.基础知识：CPU主要组成

寄存器：分为指令寄存器和数据寄存器，负责暂存指令、ALU所需操作数、ALU算出结果等。



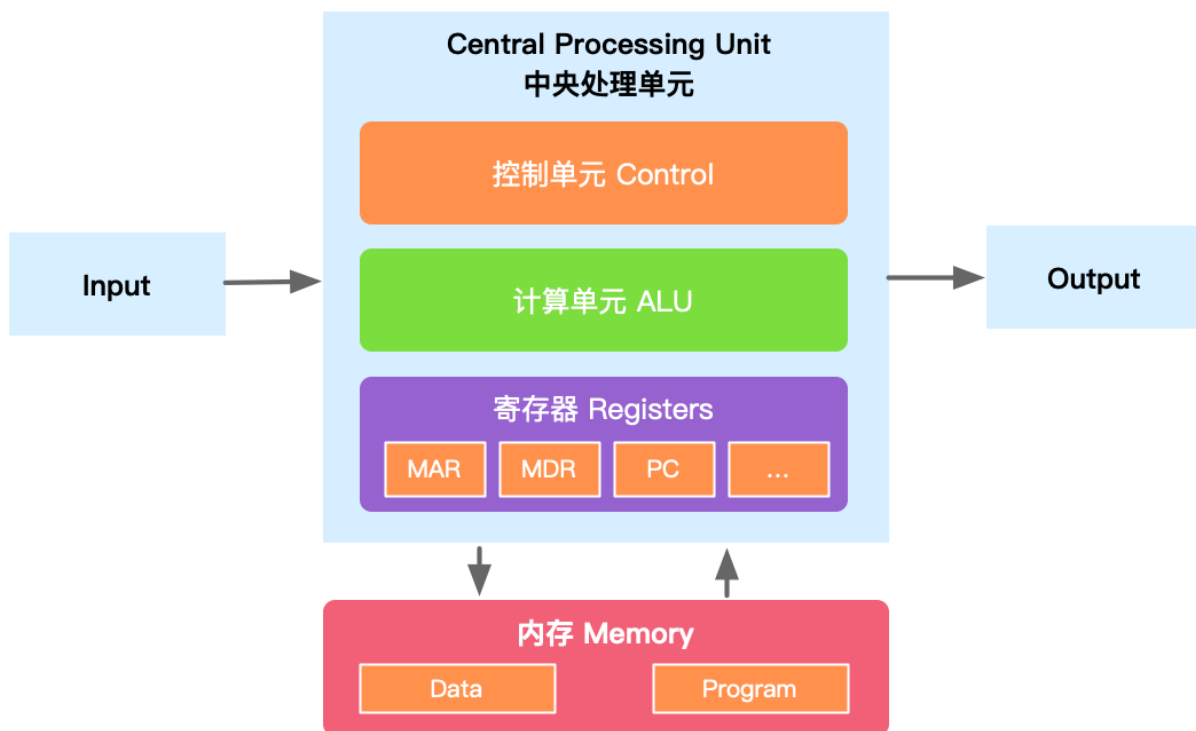
01.基础知识：CPU主要组成

控制器：负责调度工作，包括对要执行的指令进行译码、从内存中调取数据给寄存器、向运算器和寄存器发出具体操作指令等。

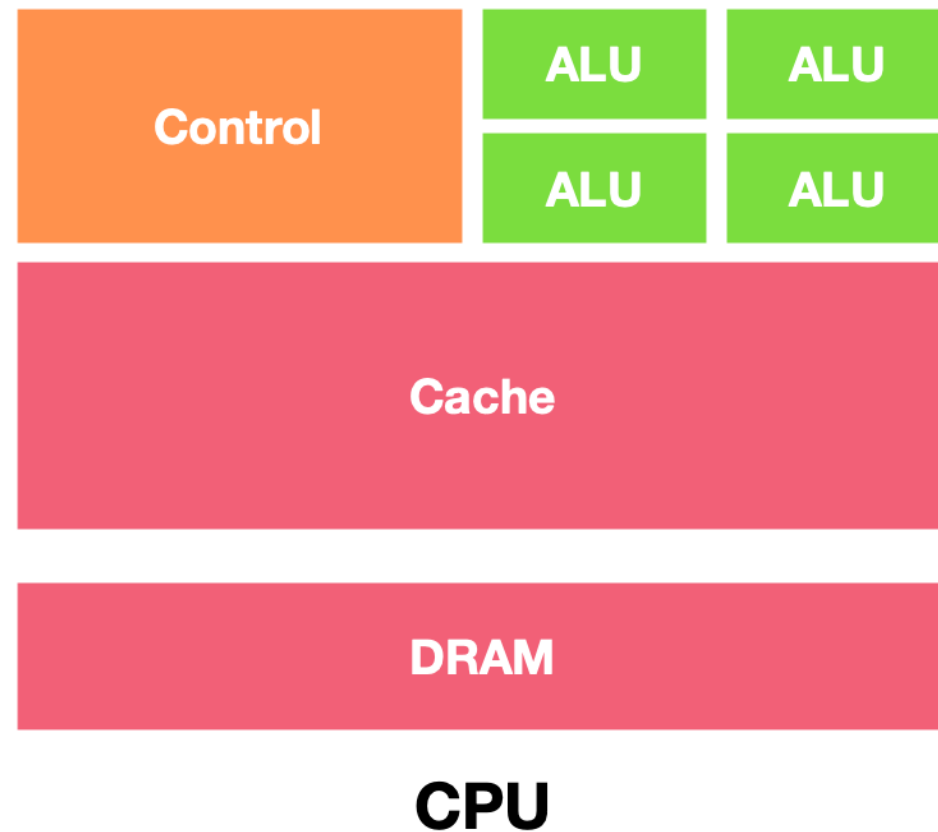


01.基础知识：CPU特点

CPU三大组成的各自分工，控制器和寄存器负责的工作最多

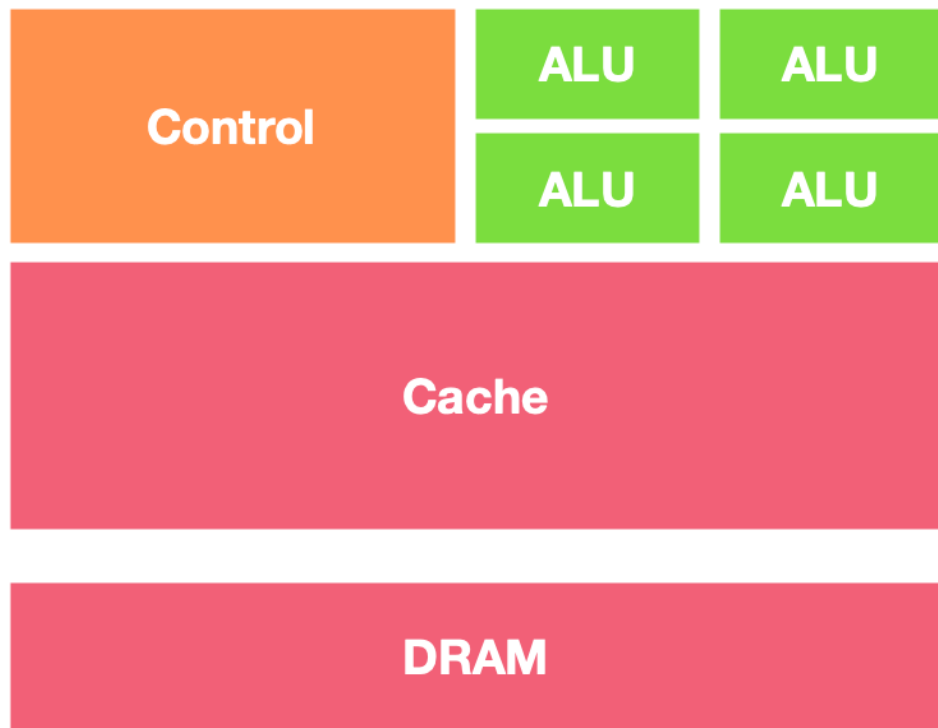


CPU架构图：主要擅长是逻辑控制，而非计算。

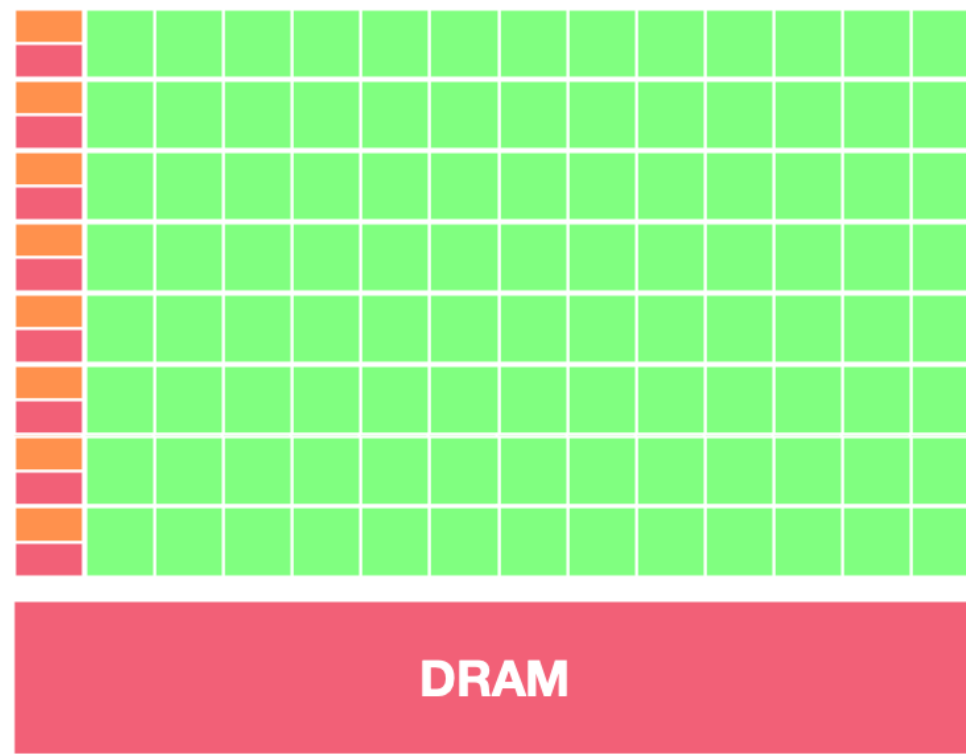


01.基础知识：CPU vs GPU

CPU: Central Processing Unit,
中央处理单元

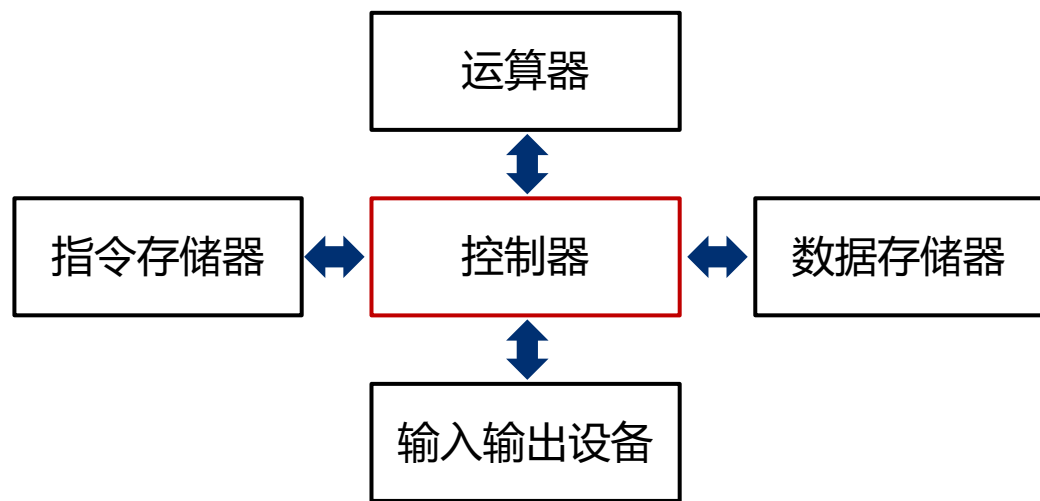


GPU: Graphics Processing Unit
图形处理单元

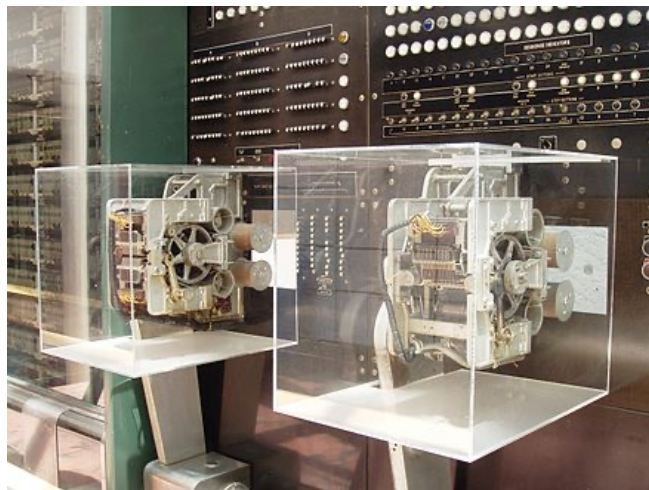


01.基础知识：计算机系统结构2—哈佛结构

- 将**程序指令储存**和**数据储存**分开的存储器结构，控制器是核心
- 提出于1936~1944间
- 世界上第一台大型自动数字计算机Mark I，采用哈佛结构



哈佛结构



Harvard Mark I



霍华德·艾肯
Howard Hathaway Aiken
1900-1973

哈佛大学

01.基础知识：哈佛结构和冯诺依曼结构的应用

当前使用哈佛结构的芯片



单片机芯片

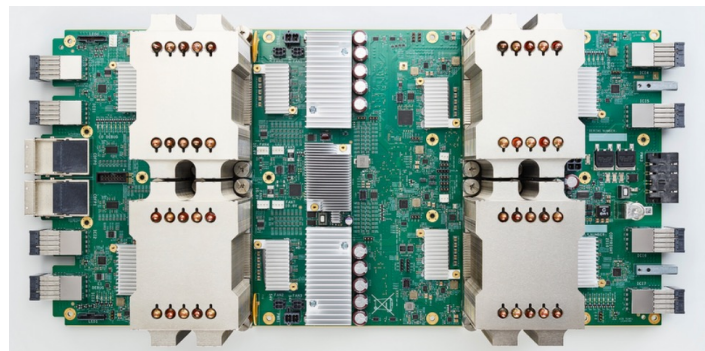


类脑计算芯片



数字信号处理芯片

当前使用冯诺依曼结构的芯片



人工智能加速器



终端CPU芯片



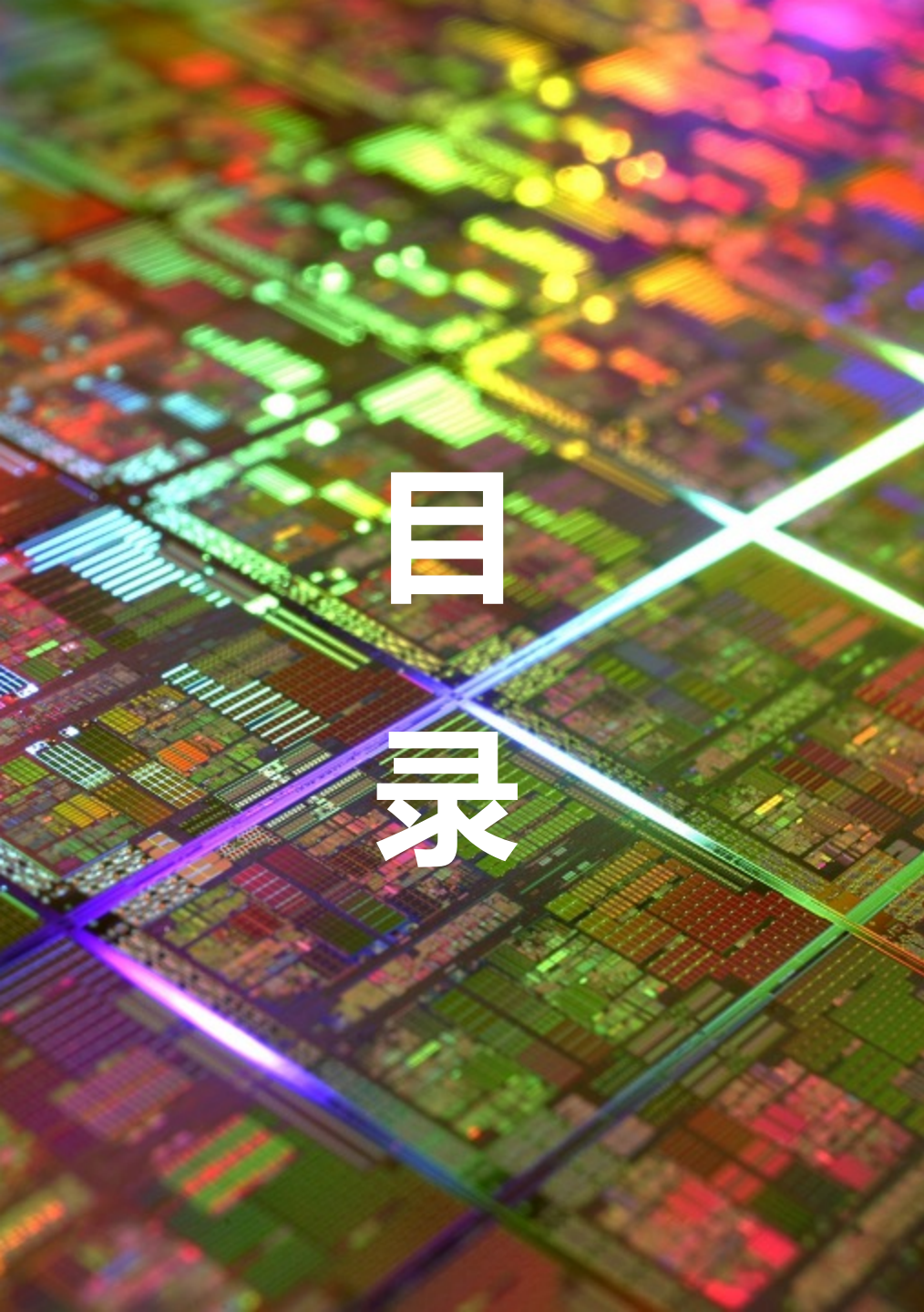
图形加速器

计算机体系结构的发展并非简单的一种结构完全替代另一种结构！

01.基础知识：“中国计算机之母”——夏培肃



1945年大学毕业后



目 录

01

基础知识

02

CPU指令集

03

存储设备

04

操作系统

02. CPU指令集：定义

计算机语言中的基本单词称为**指令**，一台计算机的全部指令称为该计算机的**指令集** (Instruction Set) 。

机器语言 (Machine Language)

- 用二进制代码表示，计算机能直接识别和执行的一种机器指令的集合（是计算机设计者通过计算机的硬件结构赋予计算机的操作功能）；
- 计算机最原始的语言，计算机在工作的时候只认识机器语言，即0和1的代码；
- 一般由操作码和操作数两部分组成。
- **注：**不同型号的计算机其机器语言是不相通的，按着一种计算机的机器指令编制的程序，不能在另一种计算机上执行。

02. CPU指令集：汇编语言

汇编语言 (Assembly Language)

- 用人类容易理解和记忆的字母、单词来代替一个特定的指令（用字母或者单词来表示一组0和1的代码），如用“ADD”来表示“0011”，用“R1”来表示“001”。
- 用助记符代替机器指令的操作码，用地址符号或标号代替指令或操作数的地址。
- 在不同设备中，对应着不同的机器语言指令集，通过汇编过程转换成机器指令。
- 特定的汇编语言和特定的机器语言指令集一一对应，不同平台之间不可直接移植。

Machine code	Assembly code	Description
001 1 000010	LOAD #2	Load the value 2 into the Accumulator
010 0 001101	STORE 13	Store the value of the Accumulator in memory location 13
001 1 000101	LOAD #5	Load the value 5 into the Accumulator
010 0 001110	STORE 14	Store the value of the Accumulator in memory location 14
001 0 001101	LOAD 13	Load the value of memory location 13 into the Accumulator
011 0 001110	ADD 14	Add the value of memory location 14 to the Accumulator
010 0 001111	STORE 15	Store the value of the Accumulator in memory location 15
111 0 000000	HALT	Stop execution

02. CPU指令集：一个简单的计算机指令系统

ADD R, M

- 功能：将R的内容与M中的内容相加后存入R

运算类指令

LOAD R, M

- 功能：将M中的内容装入R

传送类指令

STORE M, R

- 功能：将R的内容存入M中

JMP L

- 功能：无条件转向L处

转移类指令

*注：M和L为存储器地址，R为寄存器编号（寄存器是CPU内部的小型存储区域）

02. CPU指令集：一个简单的计算机指令系统

每条指令等长，均为2个字节

第一个字节的高4位是操作码

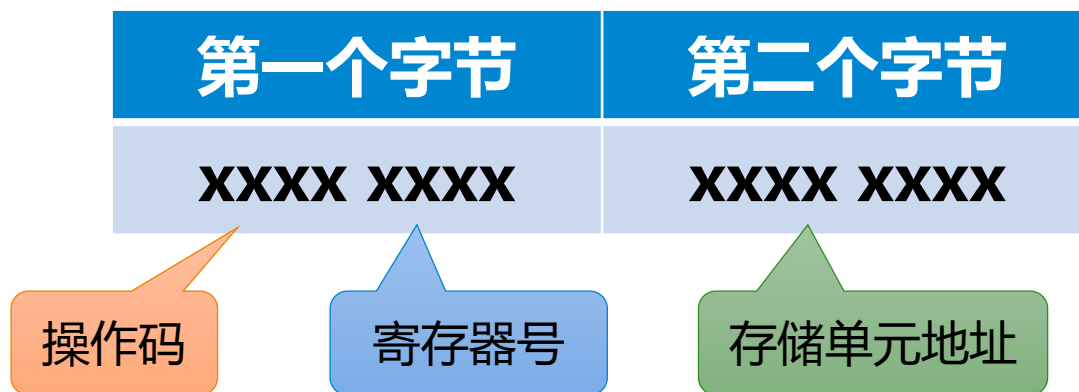
- LOAD: 0000; ADD: 0001
- STORE: 0010; JMP: 0011
- 目前只提供4条指令，最多可扩展到16条

第一个字节的低4位是寄存器号

- R0~R3: 0000~0011
- 目前只提供4个寄存器，最多可扩展到16个

第二个字节是存储单元地址

- 最大可以使用256个字节的存储器



0001 0001	0000 1001
ADD R1, [9]	

0101 1010	0000 1001
错误指令示例	

02. CPU指令集：一个简单的计算机指令系统

任务说明

1. 将M1的内容与M2的内容相加后存入M3
2. 完成运算后，程序转向L处的指令继续执行

* M1、M2、M3和L均为存储单元的地址

程序描述

1. 将M1的内容送入某个寄存器，记为Rx
2. 将Rx的内容与M2的内容相加，运算结果存入Rx
3. 将Rx的内容送入M3中
4. 转移到L，取出下一条指令继续执行

可用的指令

ADD R, M

LOAD R, M

STORE M, R

JMP L

02. CPU指令集：一个简单的计算机指令系统

运算任务对应的程序（设：M1=5，M2=6，M3=7，L=18）

汇编语言程序	机器语言程序	程序的功能
LOAD R3, [5]	00000011 00000101	将存储单元[5]的内容送入寄存器R3
ADD R3, [6]	00010011 00000110	将寄存器R3的内容加上存储单元[6]的内容，再送回R3
STORE [7], R3	00100011 00000111	将寄存器R3的内容送入存储单元[7]中
JMP [18]	00110000 00010010	转向存储单元[18]，取出指令继续执行

02. CPU指令集：一个简单的计算机指令系统

存储器地址	存储器内容	说明
0000 0101	00001100	地址 [5]，存放了数据12
0000 0110	00100010	地址 [6]，存放了数据34
0000 0111	00000000	地址 [7]，准备存放运算结果
0000 1000	00000011	“LOAD R3, [5]”的第一个字节
0000 1001	00000101	“LOAD R3, [5]”的第二个字节
0000 1010	00010011	“ADD R3, [6]”的第一个字节
0000 1011	00000110	“ADD R3, [6]”的第二个字节
0000 1100	00100011	“STORE [7], R3”的第一个字节
0000 1101	00000111	“STORE [7], R3”的第二个字节
0000 1110	00110000	“JMP [18]”的第一个字节
0000 1111	00010010	“JMP [18]”的第二个字节
0001 0000		第五条指令的第一个字节
0001 0001		第五条指令的第二个字节
0001 0010		第六条指令的第一个字节（地址 [18]）

02. CPU指令集：分类

复杂指令集（Complex Instruction Set Computer, CISC）：使用大量的指令，包括复杂指令。如x86。

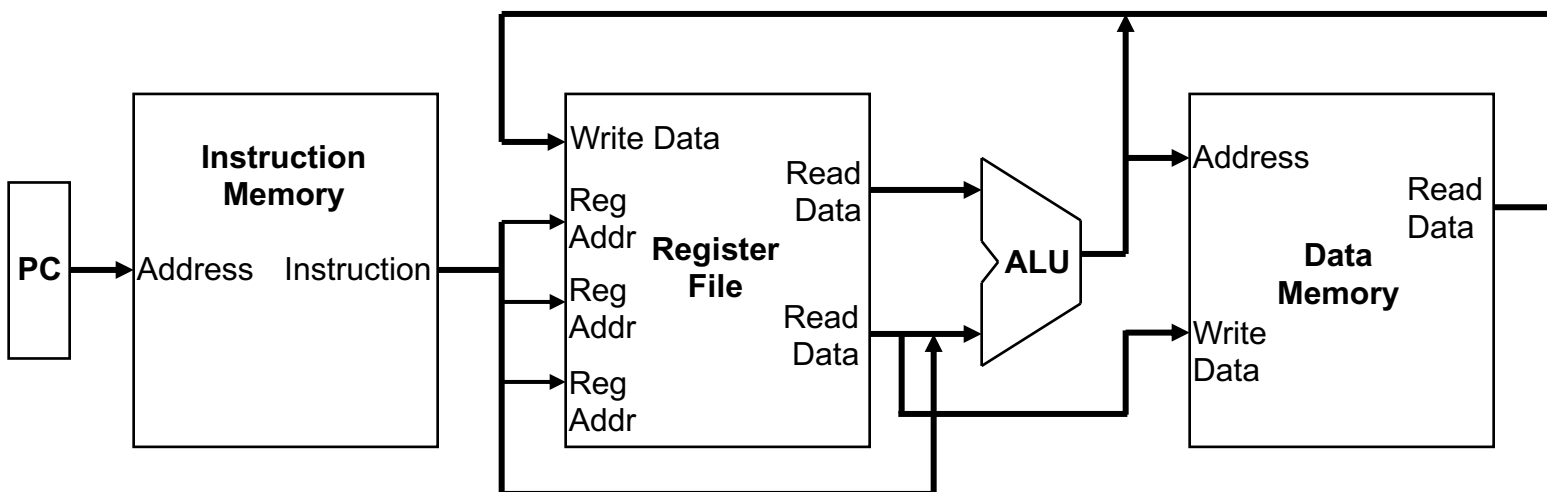
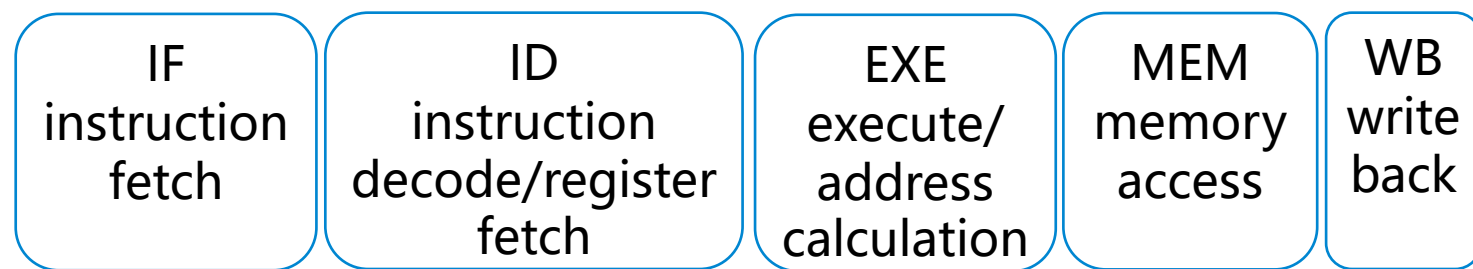
精简指令集（Reduced Instruction Set Computer, RISC）：使用少量的指令完成最少的简单操作。如ARM，MIPS，RISC-V。

比较内容	CISC	RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般大于 200	一般小于 100
指令格式	一般大于 4	一般小于 4
寻址方式	一般大于 4	一般小于 4
指令字长	不固定	等长
可访存指令	不加限制	只有取数/存数指令
各种指令使用频率	相差很大	相差很大
各种指令执行时间	相差很大	绝大多数在一个周期内完成

02. CPU指令集：指令的执行过程

通常，执行一条指令，包含如下5个处理步骤：

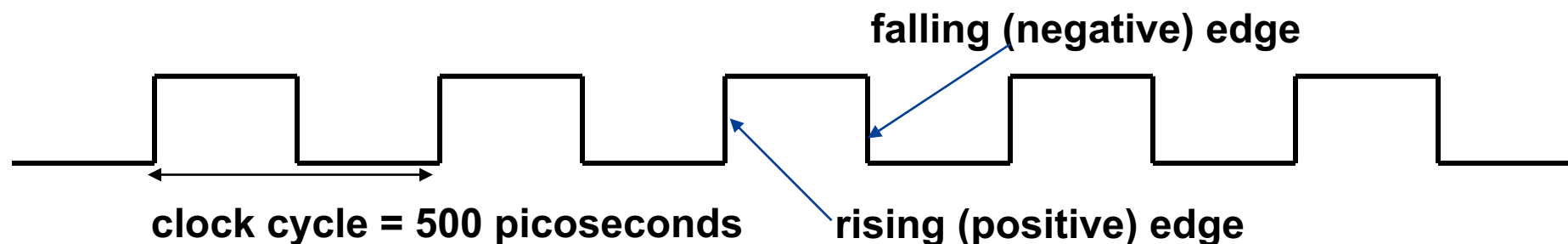
- 取指 (IF)
 - ✓ 从指令存储器中读取指令
- 译码 (ID)
 - ✓ 指令译码的同时读取寄存器
- 执行 (EXE)
 - ✓ 执行操作或计算地址
- 访存 (MEM)
 - ✓ 从数据存储器中读取操作数
- 写回 (WB)
 - ✓ 将结果写回寄存器



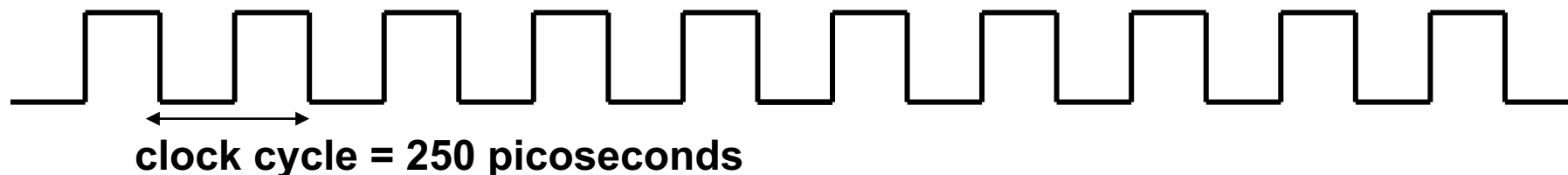
02. CPU指令集：时钟周期与时钟频率

几乎所有计算机都用时钟来驱动硬件中发生的各种事件：

- 时钟周期：时钟间隔的时间
- 时钟频率：时钟周期的倒数
- 例如：CPU A 的时钟周期为 500 皮秒 (picoseconds) ，对应的时钟频率为2GHz



- CPU B 的时钟周期为 250 picoseconds, 对应的时钟频率为4GHz



02. CPU指令集：性能的度量

一个程序需要的CPU执行时间（简称“CPU时间”）：

- $\text{CPU时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期} = \text{指令数} \times \text{CPI} / \text{时钟频率}$

指令数：执行该程序所需的总指令数量。

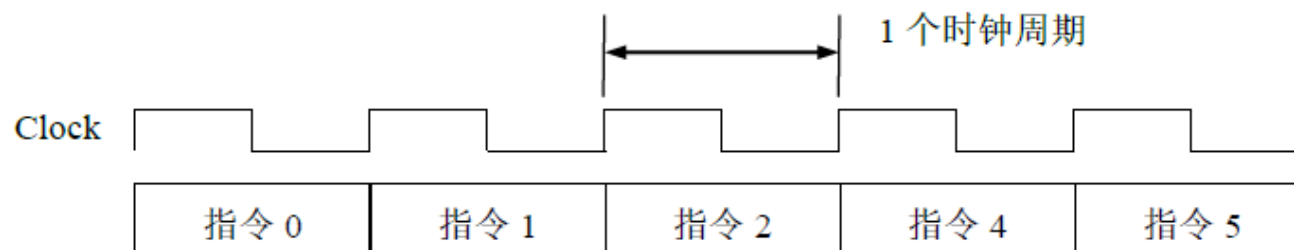
CPI（Clock Cycle per Instruction）：每条指令的时钟周期数。

- 表示执行某个程序或者程序片段时每条指令所需的时钟周期的平均数。

02. CPU指令集：CPU的实现方式

单周期

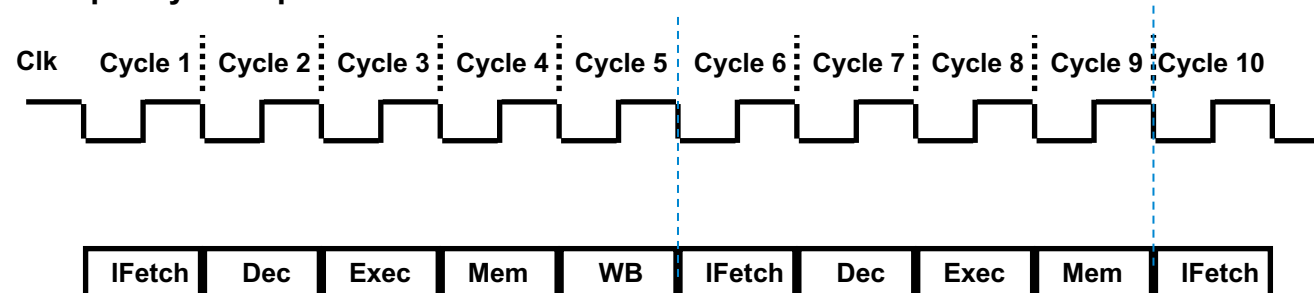
- 在一个时钟周期内完成一条指令的所有的工作。
- 即，从指令取出，到得到结果，全部在一个时钟之内完成。



多周期

- 一条指令的执行需要经过多个时钟周期。
- 执行不同指令所需要的时钟周期数可能不同。

Multiple Cycle Implementation:

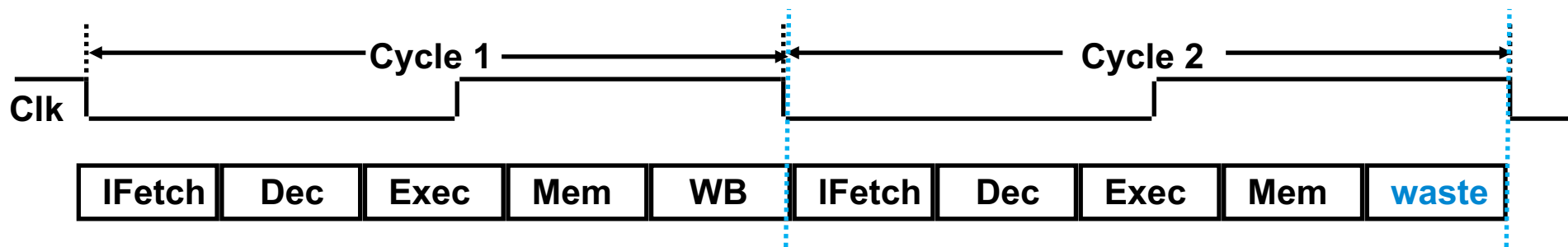


02. CPU指令集：单周期CPU的优缺点

优点：简单，容易理解。

缺点：在现实中，由于太慢而不实用

- 单周期设计中每条指令的时钟周期必须相同，时钟周期要由执行时间最长的那条指令决定。
- 对于更复杂的指令，比如浮点乘法，问题就更大了。
- 如果我们等待一条指令完全执行完成了以后再执行下一条指令，会导致硬件资源的浪费：
 - ✓ 指令的执行过程将是5个步骤依次执行。当指令执行到某个步骤时，剩下4个步骤的硬件资源将被闲置。



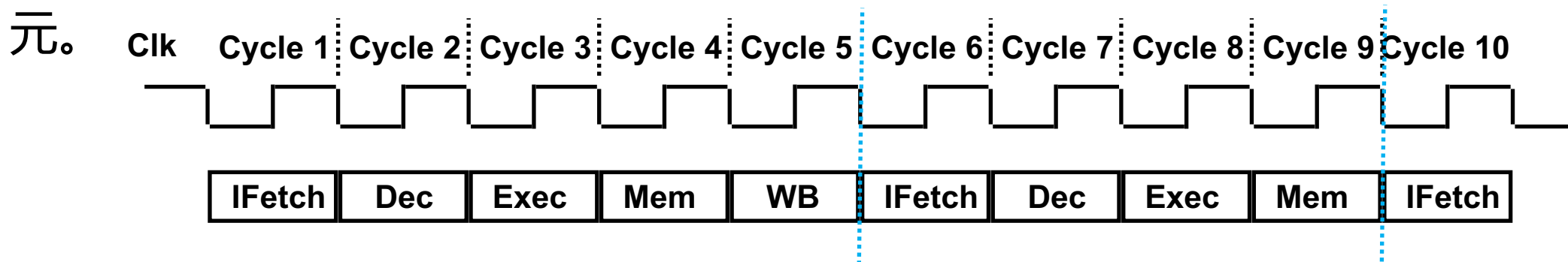
02. CPU指令集：多周期CPU的优缺点

优点：更有效地使用时钟周期——时钟周期由最慢的指令步骤决定。

多周期实现允许：

- ✓ 更快的时钟频率。
- ✓ 不同的指令采取不同数量的时钟周期数。
- ✓ 指令可以在不同的时钟周期内多次使用同一个功能单元。

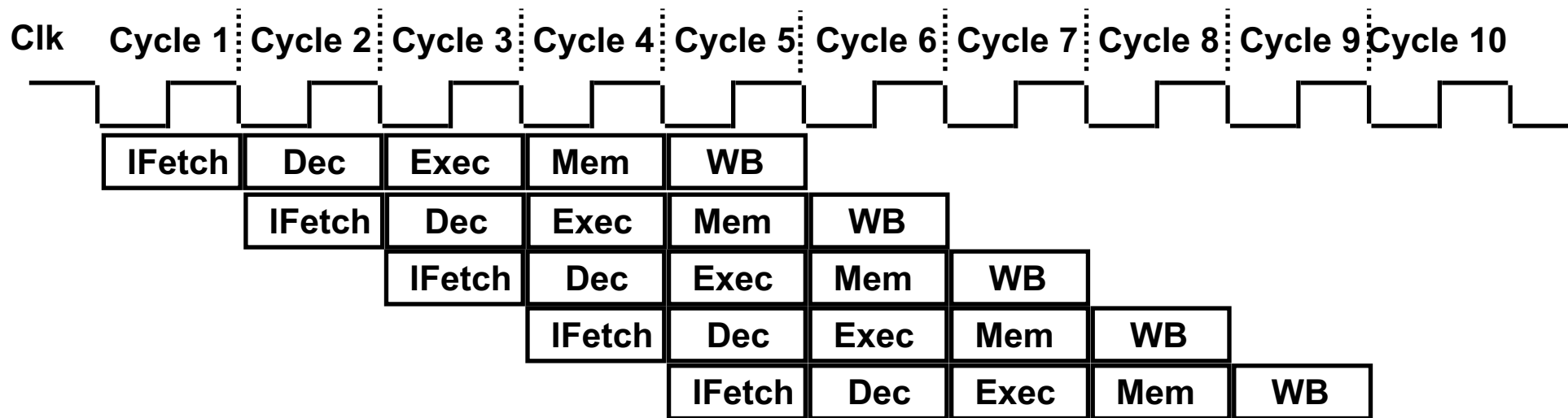
缺点：CPU的实现更为复杂，需要额外的内部状态寄存器，和更复杂的控制单元。



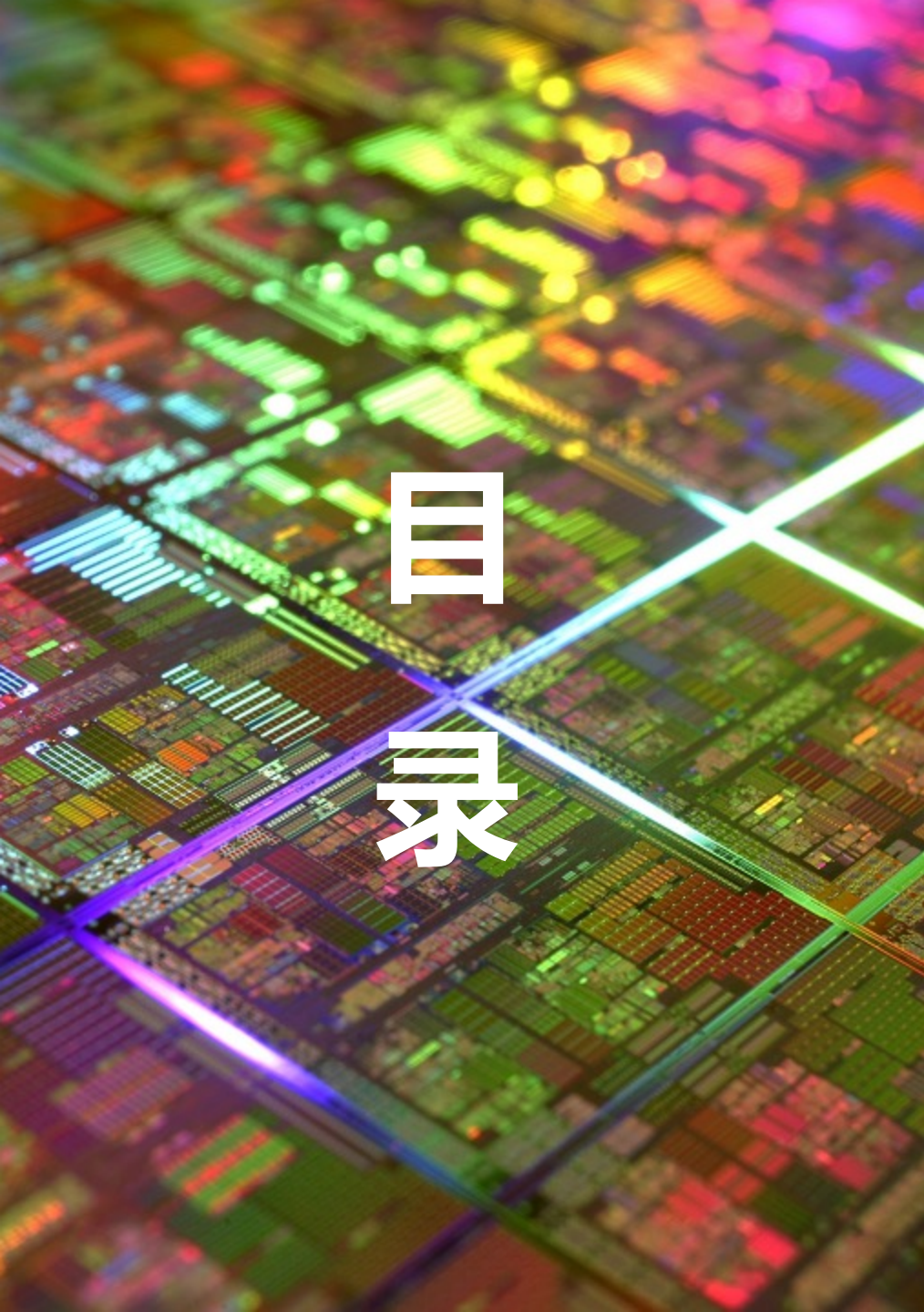
02. CPU指令集：流水线

流水线

- 在多周期CPU设计的基础上，利用各阶段电路间可并行执行的特点，让各个阶段的执行在时间上重叠起来，这种技术就是流水线技术。



- 流水线速度是原来速度的 5 倍



目 录

01

基础知识

02

CPU指令集

03

存储设备

04

操作系统

03. 存储设备：存储层次

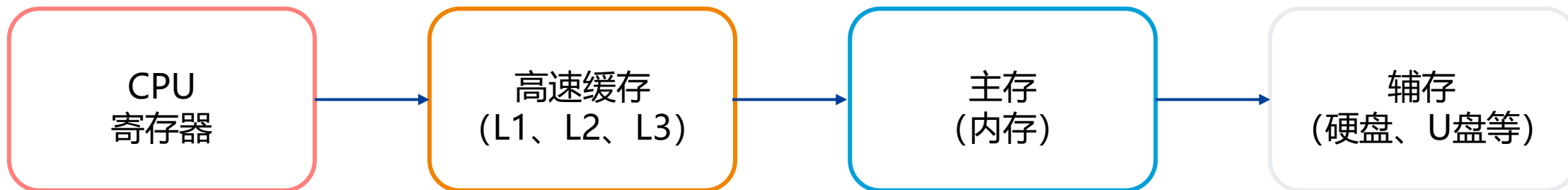
存储系统：

- 指计算机中由存放程序和数据的各种存储设备、控制部件及管理信息调度的设备（硬件）和算法（软件）所组成的系统。

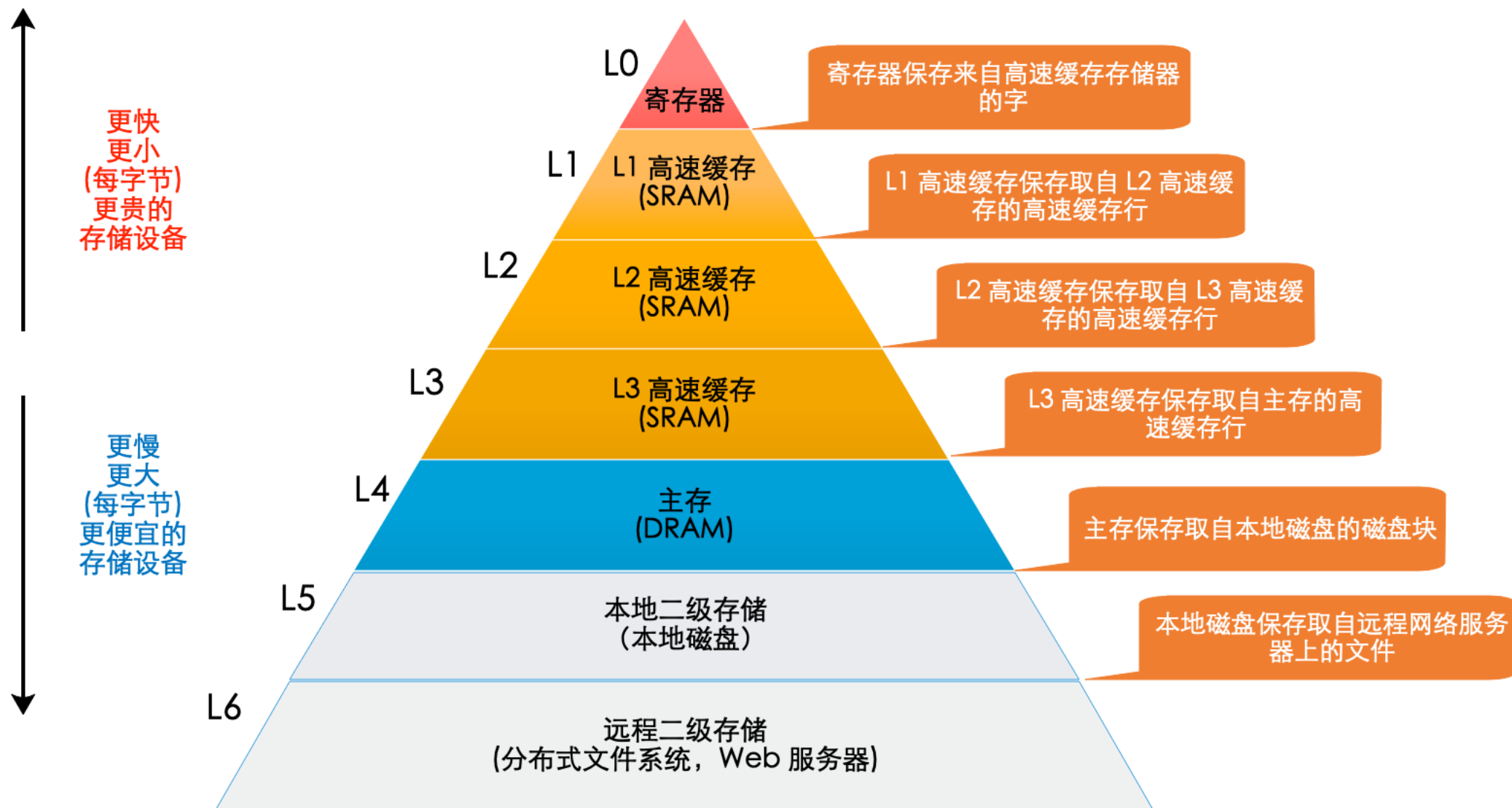
存储层次：

- 是在计算机体系结构下存储系统层次结构的排列顺序。
- 每一层于下一层相比都拥有较高的速度和较低延迟性，以及较小的容量。

对于通用计算机而言，存储层次分为四层：



03. 存储设备：存储器层次结构



03. 存储设备：存储设备-1

寄存器 (Register) :

- 是CPU内部用来存放数据的一些小型存储区域，用来暂时存放参与运算的数据和运算结果。
- 包括通用寄存器、专用寄存器和控制寄存器。

随机存取存储器 (Random Access Memory, RAM)

- RAM工作时可以随时从任何一个指定的地址写入（存入）或读出（取出）信息。
- 一旦断电所存储的数据将随之丢失。

静态随机存取存储器 (Static Random Access Memory, SRAM) :

- 所谓的“静态”，是指这种存储器只要保持通电，里面储存的数据就可以恒常保持。

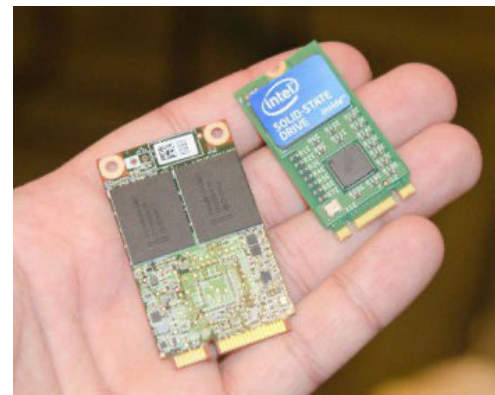
动态随机存取存储器 (Dynamic Random Access Memory, DRAM) :

- 所储存的数据需要周期性地刷新。

03. 存储设备：存储设备-2

闪存 (Flash Memory)

- 是一种非易失性存储器，即断电数据也不会丢失。
- 是一种电子式可清除程序化只读存储器的形式，允许在操作中被多次擦或写的存储器。
- 支持随机存取，比磁盘快100~1000倍。



磁盘 (Magnetic Disk)

- 是一种非易失性存储器，即断电数据也不会丢失。
- 早期计算机使用的磁盘是软磁盘 (Floppy Disk, 简称软盘)，如今常用的磁盘是硬磁盘 (Hard Disk, 简称硬盘)。



03. 存储设备：存储技术比较

Static RAM (SRAM)

- 0.5ns – 2.5ns, \$2000 – \$5000 per GB

Dynamic RAM (DRAM)

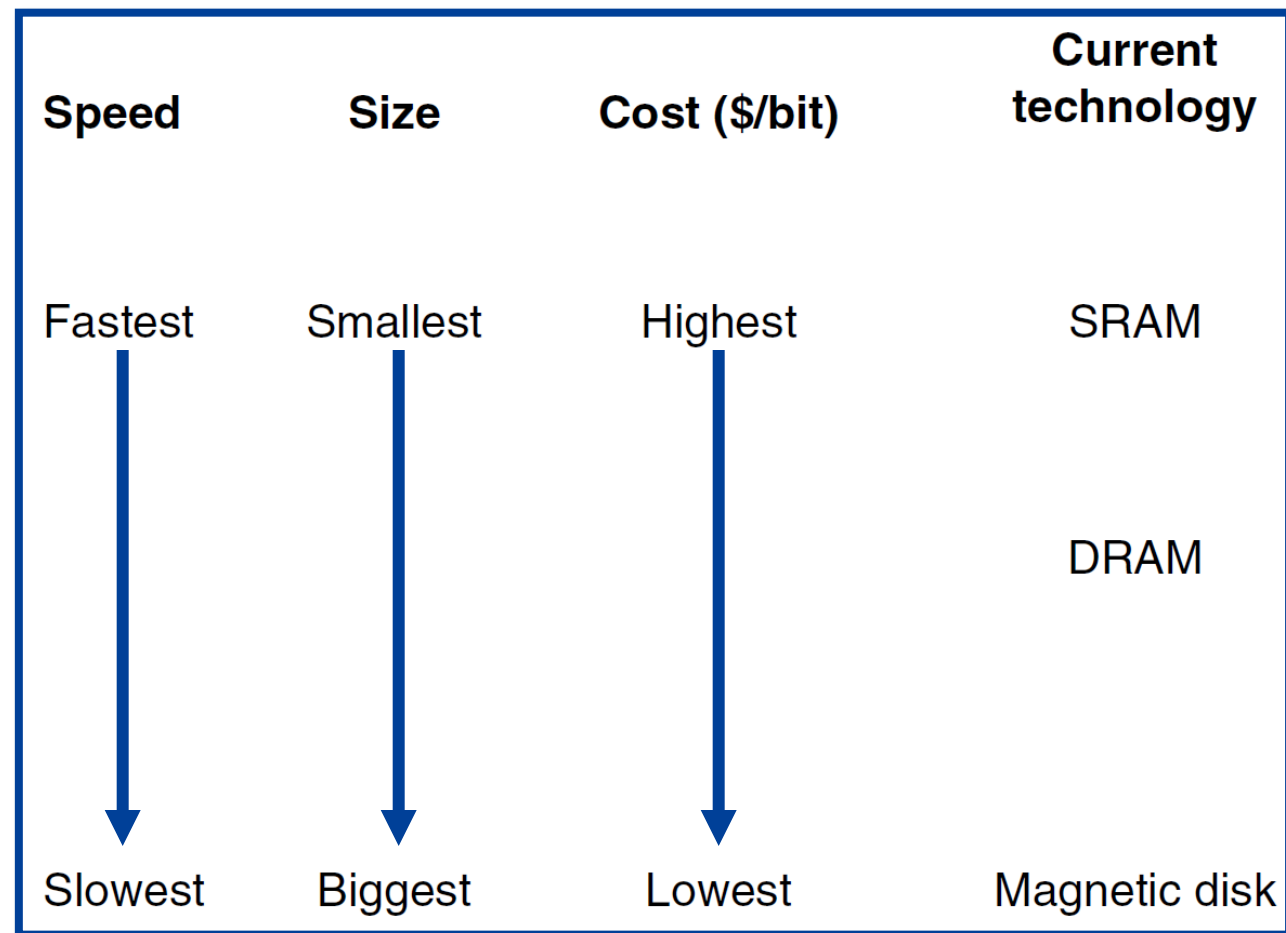
- 50ns – 70ns, \$20 – \$75 per GB

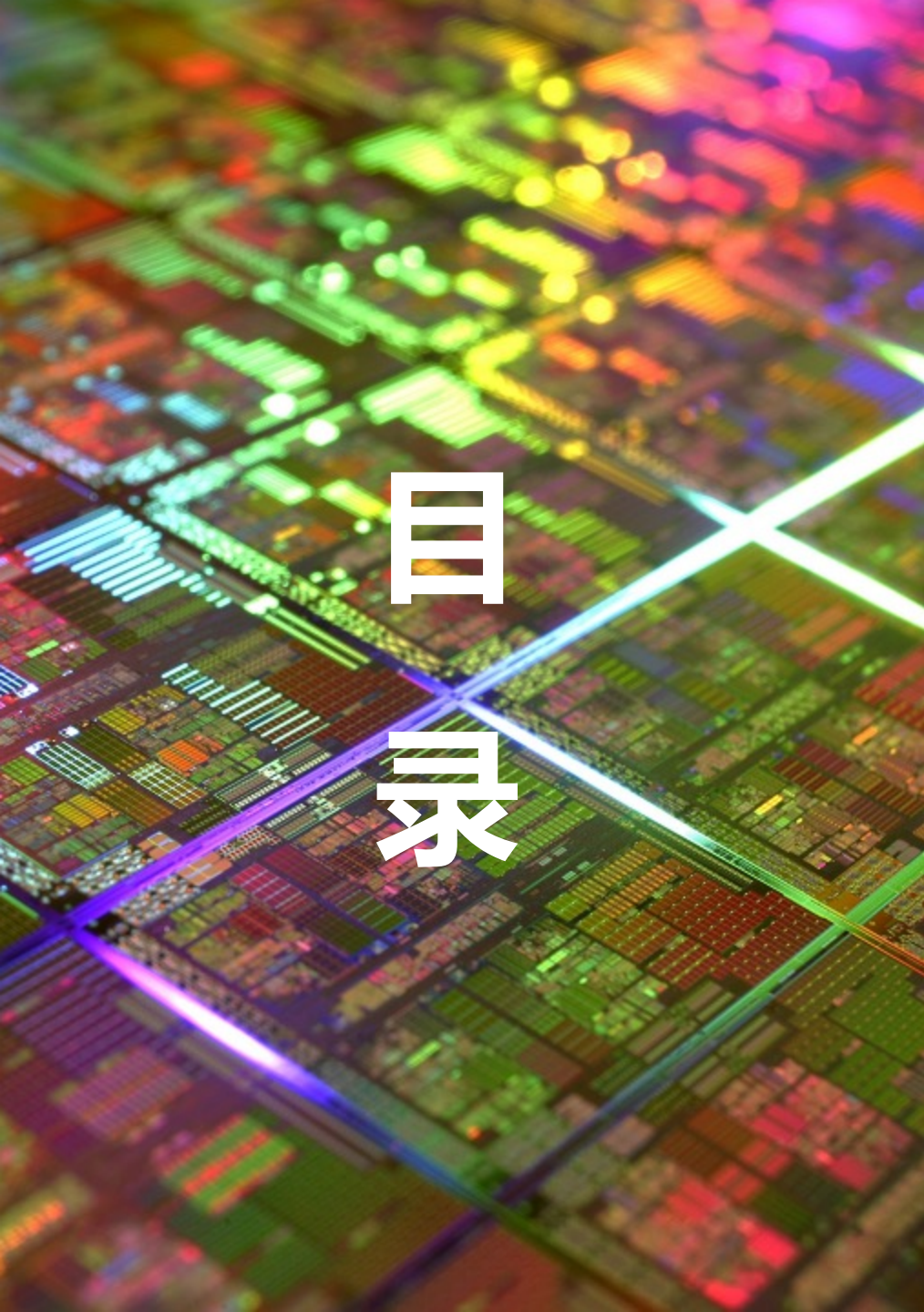
Magnetic disk

- 5ms – 20ms, \$0.20 – \$2 per GB

Ideal memory

- Access time of SRAM
- Capacity and cost/GB of disk





目 录

01

基础知识

02

CPU指令集

03

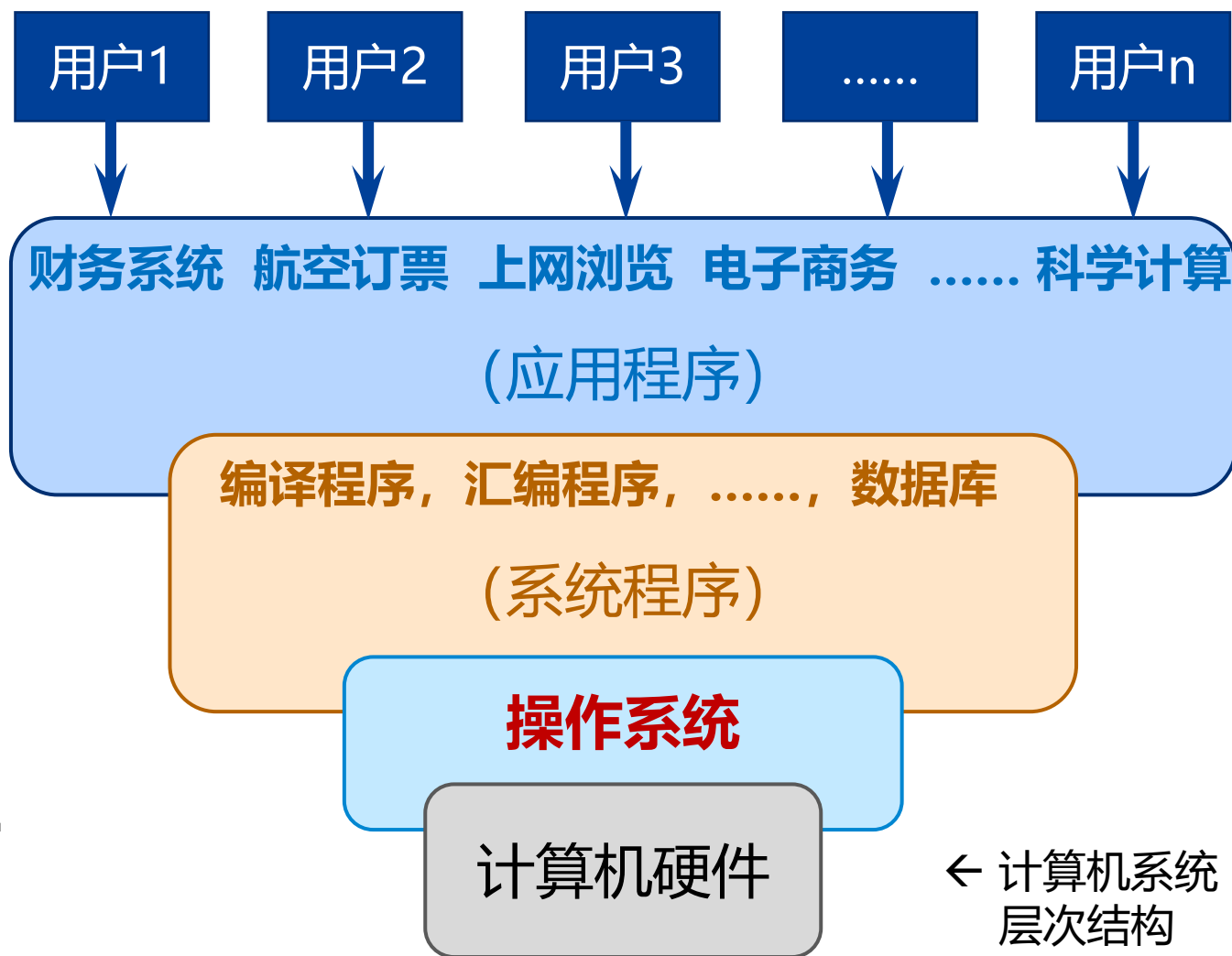
存储设备

04

操作系统

04. 操作系统：什么是操作系统？

- Operating System (OS)是
 - 有效管理计算机软硬件资源,
 - 合理组织计算机工作流程,
 - 控制程序执行,
 - 提供多种服务功能及界面,
 - 方便用户使用计算机的**系统软件**
- 是计算机中最靠近硬件的**软件**

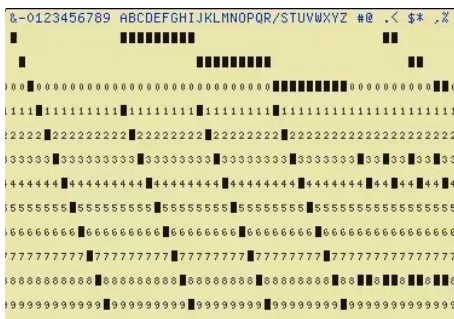


04. 操作系统：操作系统的历史

1945~1955

真空管和穿孔卡片

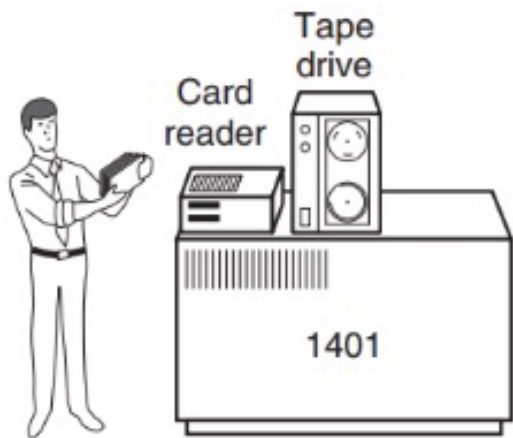
打孔的卡上面有很多排线，每条线都有可以打的孔。这些线和开关相互匹配。



1965~1980

集成电路和多道程序设计

将被用于在科学和工程工业强度数值计算的大规模科学计算机和面向字符的商用计算机进行融合。



1955~1965

晶体管 and 批处理系统

读取磁带，运行第一个工作。输出内容被写入磁带，而不是直接打印。完成一项作业后，操作系统自动载入下一项作业。

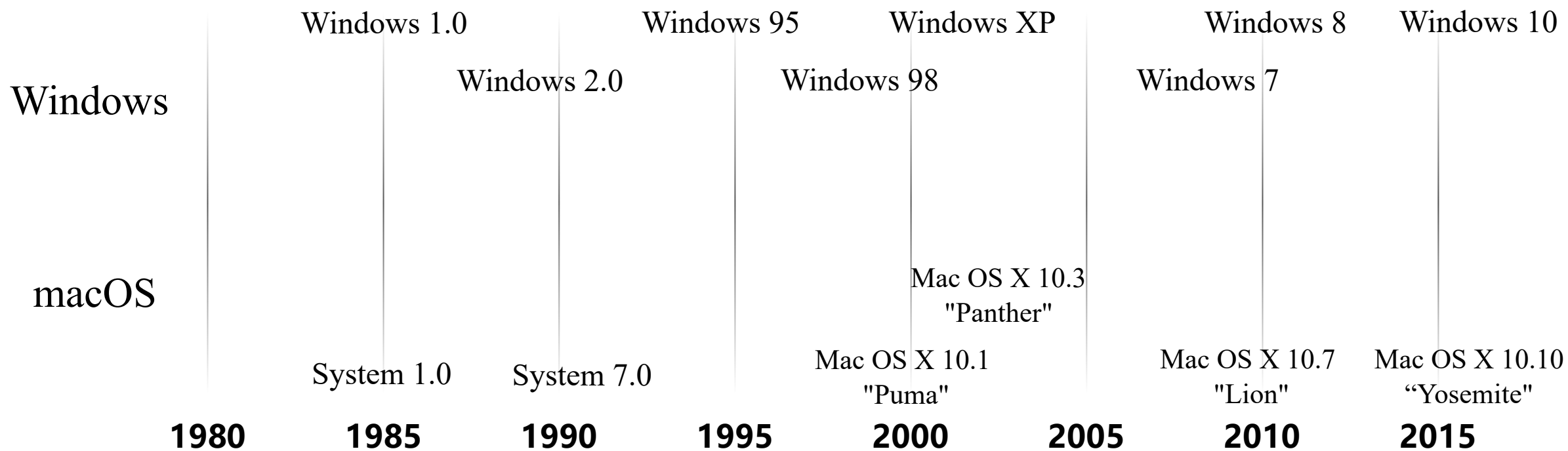
1980年至今

个人计算机

随着大规模集成电路的发展，个人电脑的时代来临了，微处理器芯片也令个人拥有电脑成为可能。



04. 操作系统： Windows与macOS的发展历程



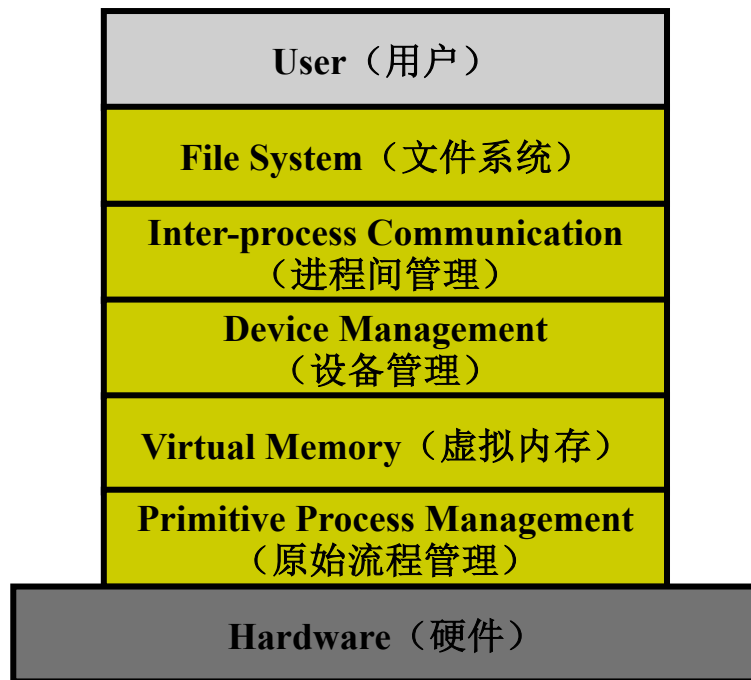
04. 操作系统：鸿蒙操作系统

- 华为鸿蒙系统（英文：HuaweiHarmonyOS）
- 2019年8月9日，正式发布操作系统鸿蒙OS。
- 鸿蒙OS是一款全场景分布式OS，可按需扩展，实现更广泛的系统安全，主要用于物联网，特点是低时延。

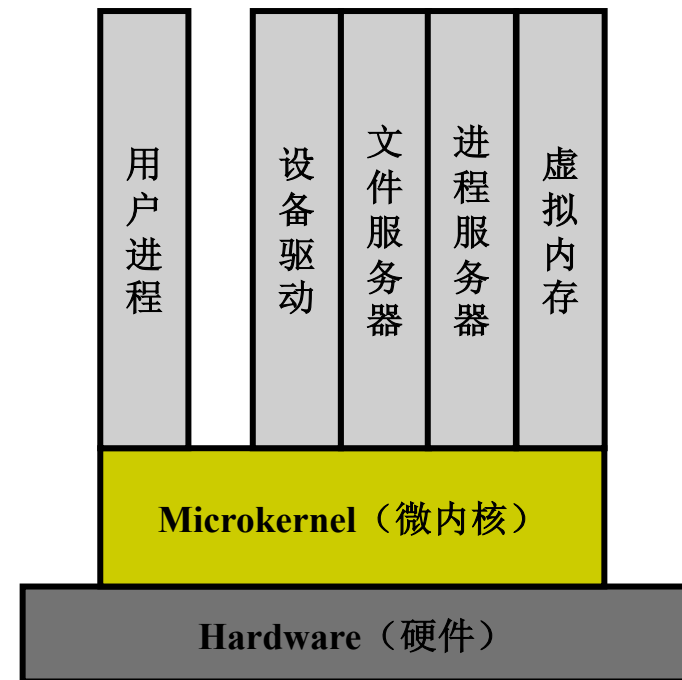


04. 操作系统：组成

- 处理器管理（进程调度）
- 存储管理
- 设备管理
- 文件管理
- 用户接口



层次化OS



微内核OS

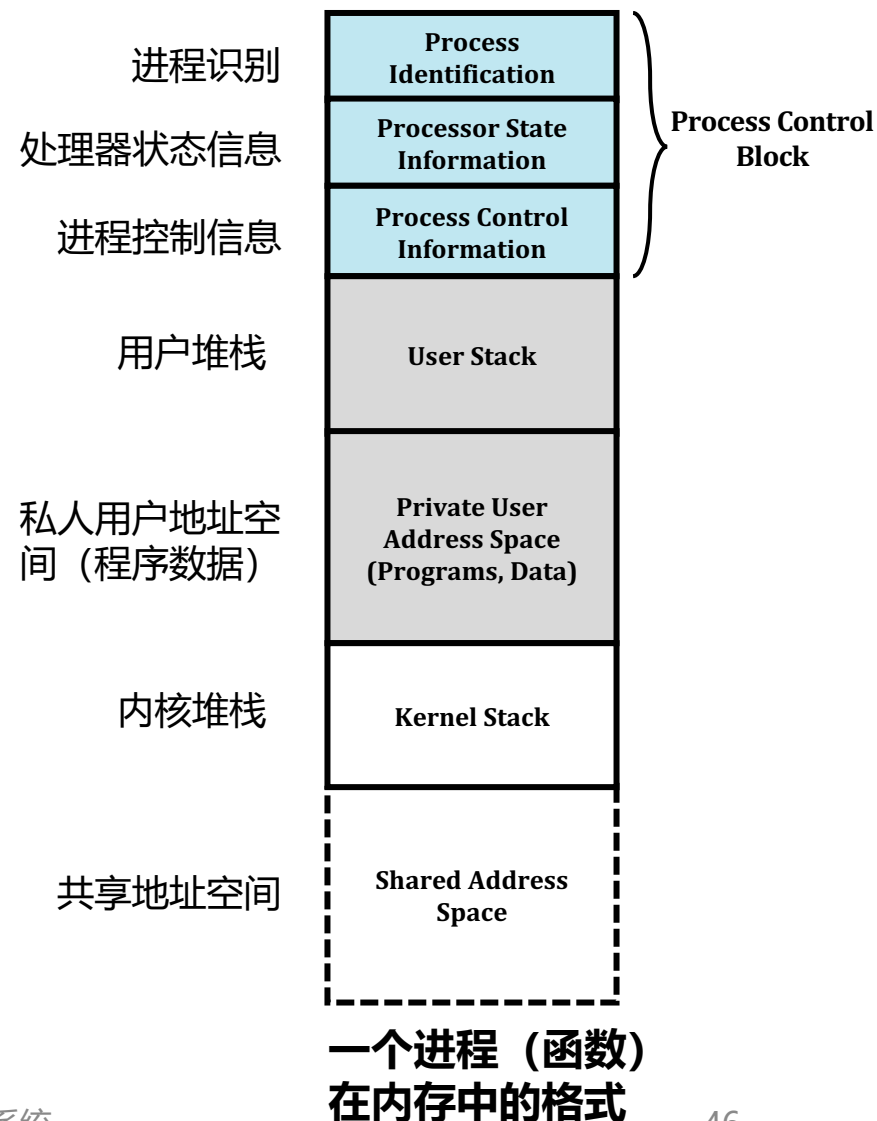
04. 操作系统：（1）处理器管理

- 在一段时间内，**只能有一个进程**在CPU中执行
- 处理器管理的基本功能：
 - 作业和进程调度
 - 进程控制
 - 进程通信：同步方式和互斥方式；通信机制

进程（Process）：一个计算机程序关于某数据集的一次执行过程 → 系统管理、分配资源基本单位

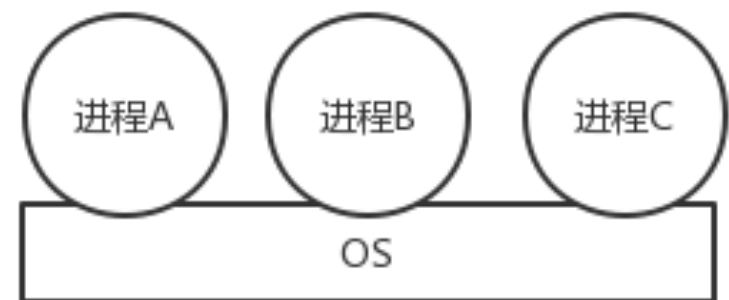
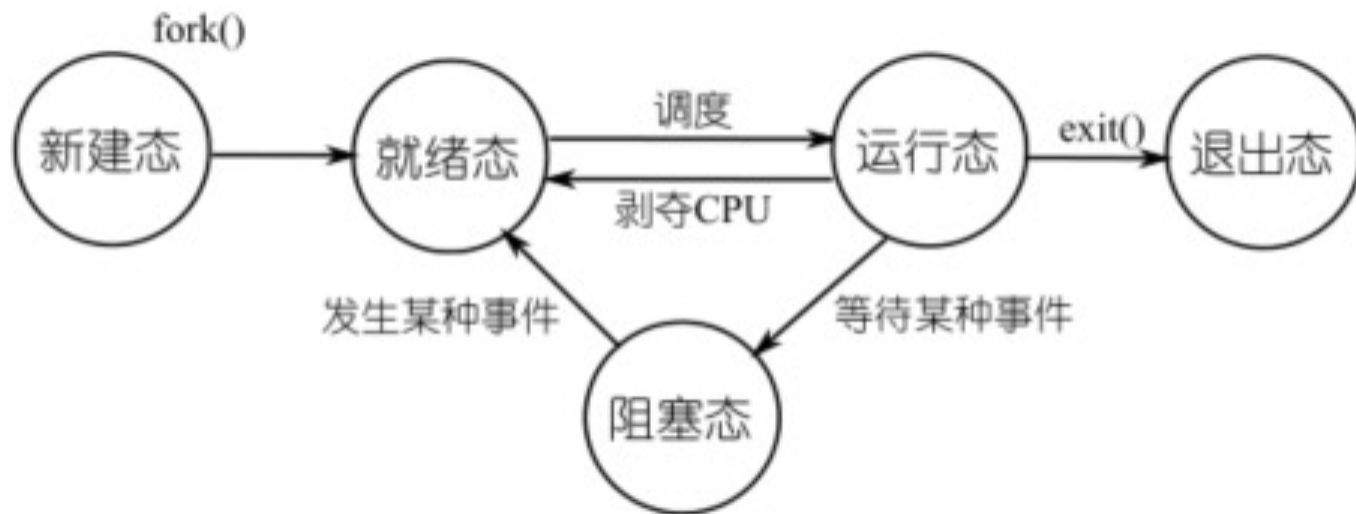
同步关系：多个进程为完成同一个任务相互协作

互斥关系：多个进程为争夺有限的系统资源进入竞争状态



04. 操作系统：进程的概念

- 程序在操作系统上的一次运行过程
- 系统进行资源分配和调度的一个可并发执行的独立单位。
- 在进程看来，自己独占计算机系统，并可以调用OS提供的服务。



04. 操作系统：进程与程序

■ 进程的特性

- 动态性：进程具有生命周期，经历创建、运行、消亡等过程。
- 并发性：多个进程可以并发地执行。
- 独立性：进程既是系统中资源分配和保护的基本单位，也是系统调度的独立单位。

■ 进程与程序关系

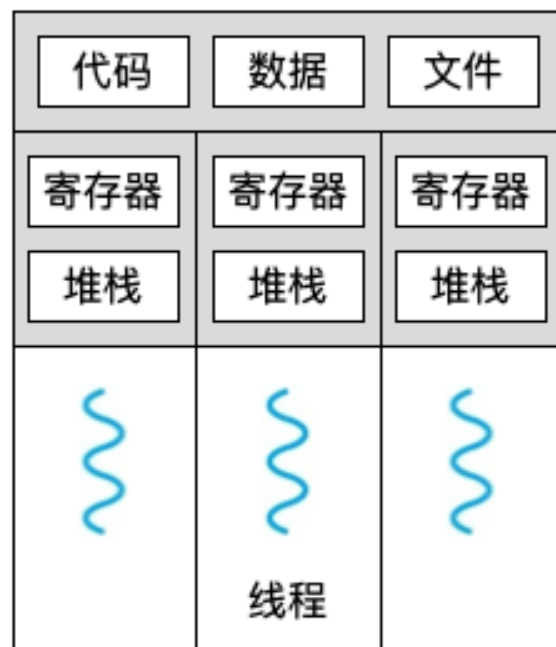
- 进程是程序的一次动态执行活动，而程序是进程运行的静态描述文本。
- 程序是一个软件，可以长期存在，而进程是一次执行过程。
- 一个程序可以对应多个进程。

04. 操作系统：进程与线程

- 线程是程序执行中一个单一的顺序控制流程，是程序执行流的最小单元，是处理器调度和分派的基本单位



单线程进程



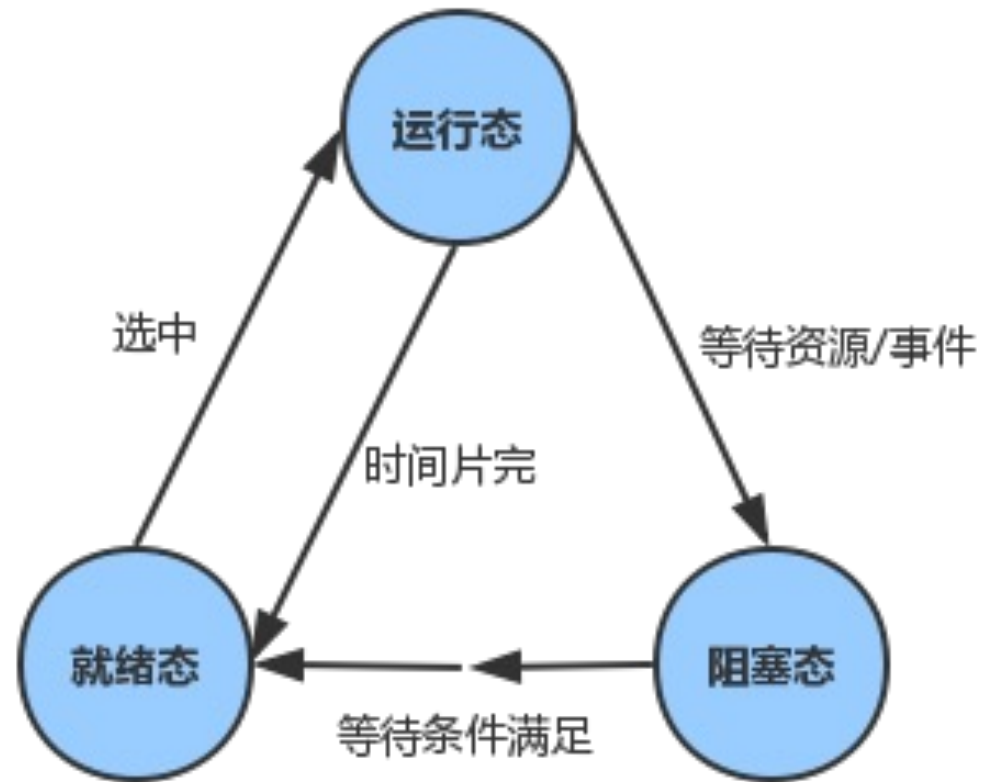
多线程进程

The screenshot shows the Windows Task Manager window with the 'Processes' tab selected. The table lists various running processes, including 'chrome.exe' and 'cmd.exe'. The '线程数' (Number of threads) column is highlighted with a red box, showing values ranging from 1 to 48. The 'PID' column is also highlighted with a red box, showing values ranging from 1380 to 13180.

映像名称	PID	用户名	CPU	内存 (专用工作集)	基本优先级	线程数	用户对象	句柄数	GDI
armsvc.exe *32	1380	SYSTEM	00	1,024 K	普通	4		75	
audiodg.exe	316	LOCAL...	02	14,272 K	普通	16		388	
BSTPopu.exe *32	6896	Admin...	00	3,568 K	普通	4	16	128	
BSTService.exe...	1424	SYSTEM	00	2,032 K	普通	5		115	
bdyyProtect.exe...	2568	Admin...	00	476 K	低于标准	3	4	180	
bdyyService.exe...	1444	SYSTEM	00	3,428 K	普通	10		324	
calrender.exe *32	11592	Admin...	00	20,800 K	普通	30	39	716	
chrome.exe *32	780	Admin...	00	4,508 K	普通	7	13	187	
chrome.exe *32	3268	Admin...	00	34,284 K	普通	22	1	153	
chrome.exe *32	4728	Admin...	00	28,848 K	低于标准	9	2	124	
chrome.exe *32	4764	Admin...	00	23,656 K	低于标准	9	2	127	
chrome.exe *32	5304	Admin...	00	166,756 K	低于标准	9	2	148	
chrome.exe *32	6564	Admin...	00	109,244 K	普通	10	3	243	
chrome.exe *32	7116	Admin...	00	179,588 K	低于标准	9	2	139	
chrome.exe *32	8272	Admin...	00	103,360 K	低于标准	9	2	138	
chrome.exe *32	8560	Admin...	00	48,304 K	低于标准	9	2	134	
chrome.exe *32	8716	Admin...	00	226,912 K	低于标准	9	2	152	
chrome.exe *32	8884	Admin...	00	128,856 K	普通	48	119	1,605	
chrome.exe *32	9056	Admin...	00	59,736 K	低于标准	9	2	128	
chrome.exe *32	9200	Admin...	00	20,108 K	普通	9	2	121	
chrome.exe *32	9948	Admin...	00	30,988 K	低于标准	9	2	130	
chrome.exe *32	10620	Admin...	00	121,972 K	低于标准	9	2	130	
chrome.exe *32	10964	Admin...	00	69,608 K	低于标准	9	2	141	
cmd.exe	12536	Admin...	00	1,384 K	普通	1	1	23	
cmd.exe	13180	Admin...	00	1,500 K	普通	1	1	25	
Computer4.LN.e...	4172	Admin...	02	1,232 K	低于标准	9	116	260	
ComputerZServ...	5432	Admin...	01	1,764 K	低于标准	14	32	245	

04. 操作系统：进程的三种典型状态

- 一个进程从创建、运行至消亡的整个生命周期，可以用一组状态加以刻画。
- 一般来说,进程在执行过程中至少经历三种不同的进程状态：
 - **运行态 (running)**：占有处理器正在运行。
 - **就绪态 (ready)**：具备运行条件，等待系统分配处理器以便运行。
 - **等待/阻塞态 (blocked)**：不具备运行条件，正在等待某个事件的发生。



04. 操作系统：状态转换事件

■ 运行态→阻塞态

- 等待资源/事件等；如等待外设传输，或人工干预。

■ 阻塞态→就绪态

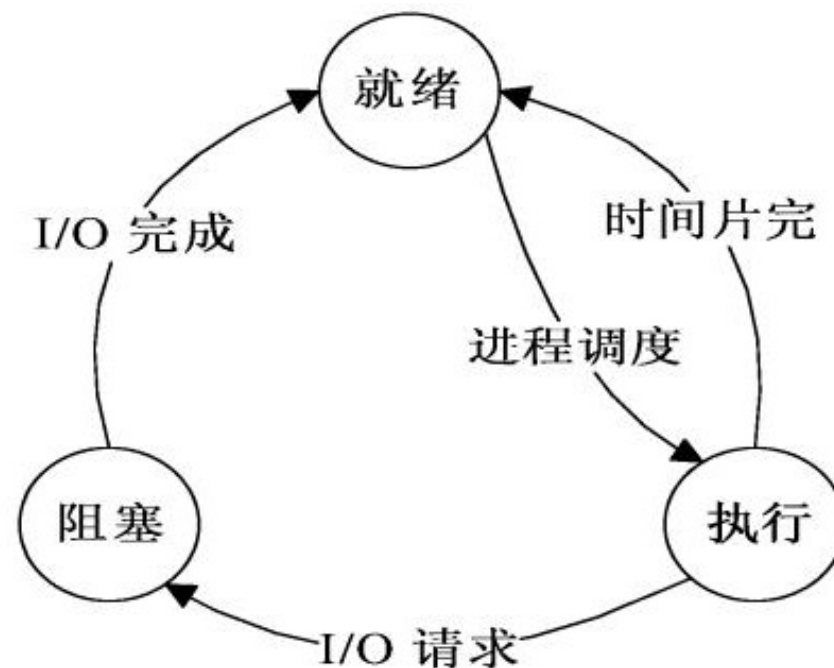
- 资源得到满足；如外设传输结束；人工干预完成。

■ 运行态→就绪态

- 运行时间片到；出现有更高优先级进程。

■ 就绪态→运行态

- 选择一个就绪进程运行。



04. 操作系统：调度算法

■ 调度准则

- 面向用户的准则
- 面向系统的准则

■ 考虑优先级

■ 调度方式

- 不可剥夺式，可剥夺式

■ 调度策略

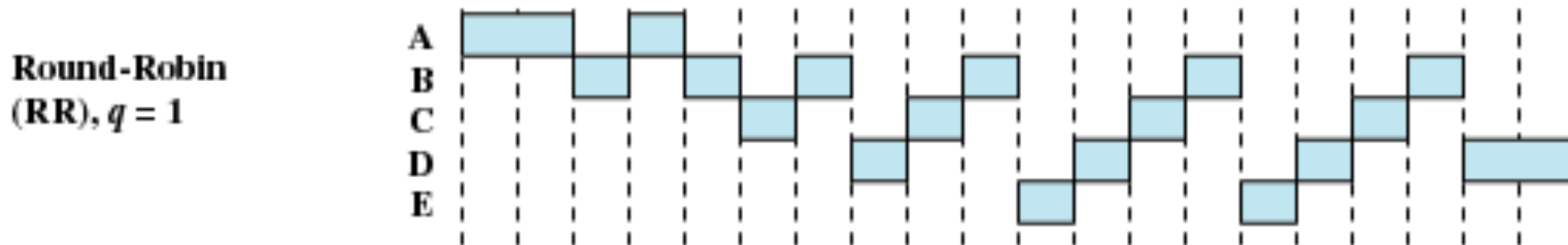
- FCFS, RR(轮转), SPN(最短进程优先), SRT(最短剩余时间优先), HRRN(最高响应比优先)等。

各调度策略的特点

	选择函数	决策模式	吞吐量	响应时间	开销	对进程影响	饿死
FCFS	Max(w)	非抢占	不强调	可能很高，特别是当进程执行时间差别很大时	最小	对短进程不利；对IO进程不利	无
RR	C	抢占（时间片）	时间片太小吞吐量很低	为短进程提供好的响应时间	与时间片大小相关	公平	无
SPN	Min(s)	非抢占	高	为短进程提供好的响应时间	可能较高	对长进程不利	可能
SRT	Min(s-e)	抢占（条件满足时）	高	提供好的响应时间	可能较高	对长进程不利	可能
HRRN	Max(R)	非抢占	高	提供好的响应时间	可能较高	很好的平衡	无

04. 操作系统：RR调度策略

- 策略思想：**各就绪进程轮流运行一小段时间，这一小段时间称为时间片。当一个进程耗费完一个时间片而尚未执行完毕，调度程序就强迫它放弃处理机，使其重新排到就绪队列末尾。



Round-Robin轮询调度算法：每一次把来自用户的请求轮流分配给系统中的服务器，从1开始，直到N(内部服务器个数)，然后重新开始循环。

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

04. 操作系统：RR调度策略详解

进程

Process	A	B	C	D	E
Arrival	0	2	4	6	8
Service	3	6	4	5	2

Round-Robin算法（按FIFO队列先进先出执行）

在t时刻时，若资源闲置则直接运行，若冲突则：

- 1. 新进程队尾入队
- 2. t-1时间片未完成进程最后入队

时刻 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

进程运行

A	1	2		3																
B			1		2		3			4				5				6		
C						1			2				3				4			
D							1					2				3			4	5
E										1					2					

FIFO队列

出队			B	A	B	C	B	D	C	B	E	D	C	B	E	D	C	B	D	
队尾入队			A	B	C	B	D	C	B	E	D	C	B	E	D	C	B	D		
							C	B	E	D	C	B	E	D	C	B	D			
									D	C	B	E	D	C	B					



04. 操作系统：进程的同步与互斥

- 两个或两个以上的进程要协作完成一个任务，它们之间就要互相配合，需要在某些动作之间进行**同步**。
- 进程间另一种关系是**互斥**，这种关系一般发生在两个或两个以上的进程竞争某些同时只能被一个进程使用的资源的情况下。
- 在一段时间内只能允许一个进程访问的资源称为**临界资源**，如打印机、磁带机、光盘刻写机、绘图仪等。
- 进程执行的访问临界资源的程序段称为**临界段**或**互斥段**。

04. 操作系统： 例子： 统计两个进程 P1和P2对共享变量count访问计数

设count初值 = 0

P1:

R1=count (0)

R1=R1+1

count=R1 (1)

P2:

R2=count (1)

R2=R2+1

count=R2 (2)

P1: R1=count (0)

R1=R1+1 (1)

P2: R2=count (0)

R2=R2+1 (1)

count=R2 (1)

P1: count=R1 (1)

结果: count = 2

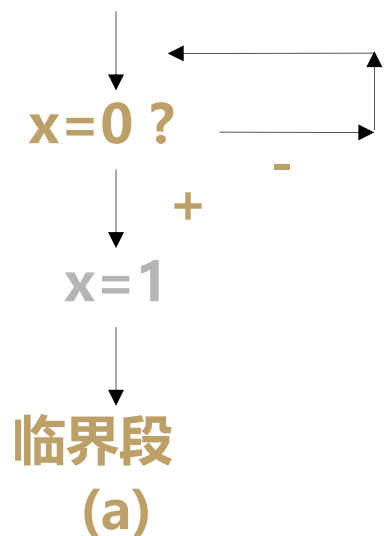
- 虽然P1和P2进程各自都执行了对count加1的操作段，但结果count只增加1。
- 因此，变量count就是临界资源，P1、P2访问count的两个程序段就是临界段，诸进程必须互斥地进入临界段。

04. 操作系统：进程间互斥控制方法

- 锁可以用于控制临界段的互斥执行。锁有两个状态，一个是打开状态，另一个是关闭状态，故锁可以用布尔变量表示。在C中，锁变量可以定义为char或int类型变量。设x为锁变量，则定义：

x = 0 锁的打开状态;
1 锁的关闭状态。

LOCK (x)



临界段

UNLOCK (x)

x=0

(b)

04. 操作系统：锁的安全控制

- 问题：锁的关闭操作包含测试和关闭两个操作，是否会被中断？
- 解决方案
 - 原子操作：专用锁操作指令，交换指令...

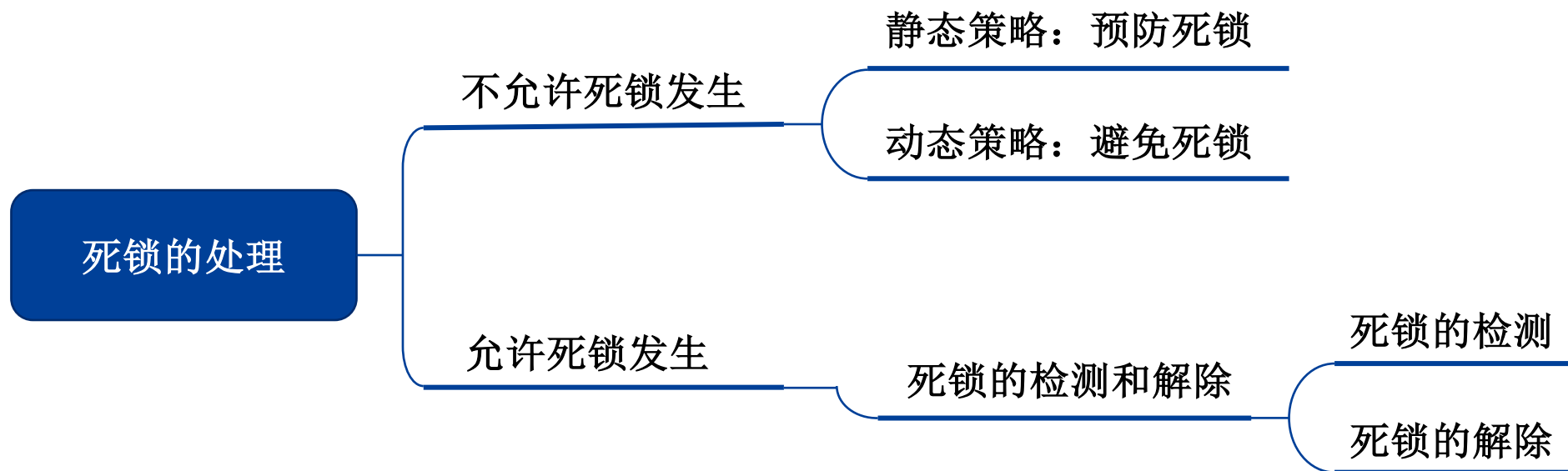
```
boolean test_and_set(boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

```
do {
    while (test_and_set(&lock))
        ; /* do nothing */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);
```


04. 操作系统：死锁

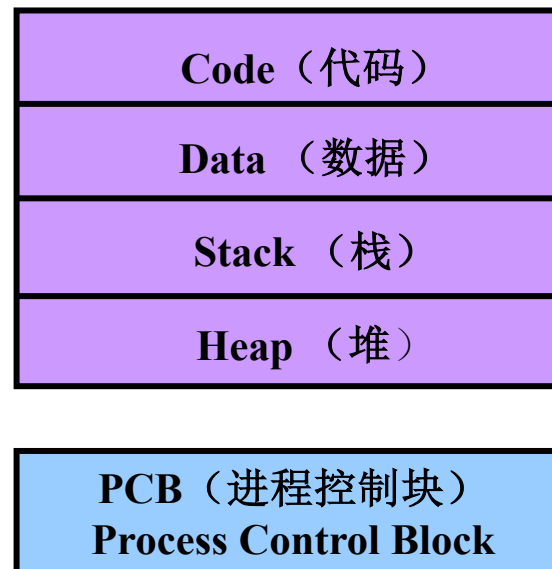
- 两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。
- 产生死锁的四大必要条件
 - ①资源互斥(Mutual exclusion): 每个资源同时只能被一个进程使用。
 - ②占有并请求(Hold and wait): 已经得到资源的进程还能继续请求新的资源。
 - ③资源不可剥夺(No preemption): 已被分配的资源只能由当前占有者主动地释放该资源。
 - ④环路等待(Circular wait): 死锁发生时，系统中存在一条环路，环路上的每个进程都在等待下一个进程所占有的资源。

04. 操作系统：死锁的处理



04. 操作系统： （2） 存储器管理

- 计算机存储器的设计目标可以归纳成三个问题： 多少？ 多快？ 多贵？
- 存储器管理的基本功能：
 - 内存分配
 - 地址映射（逻辑地址、物理地址）
 - 内存保护（上界地址值、下界地址值）
 - 内存扩充（虚拟存储技术）



04. 操作系统：（3）设备管理

- 计算机外设种类众多，而且各种设备的传输速度差异很大，很难开发一种通用的、一致的解决方案。

- 设备管理的基本功能：

- 缓冲区管理
- 设备分配
- 设备驱动
- 设备无关性

设备	传输速度	设备	传输速度
键盘	90	光盘	9000000
鼠标	200	以太网	10000000
Modem	80000	硬盘	90000000
软盘	900000	图形显示设备	700000000
激光打印机	2000000	千兆位以太网	1000000000
扫描仪	5000000		

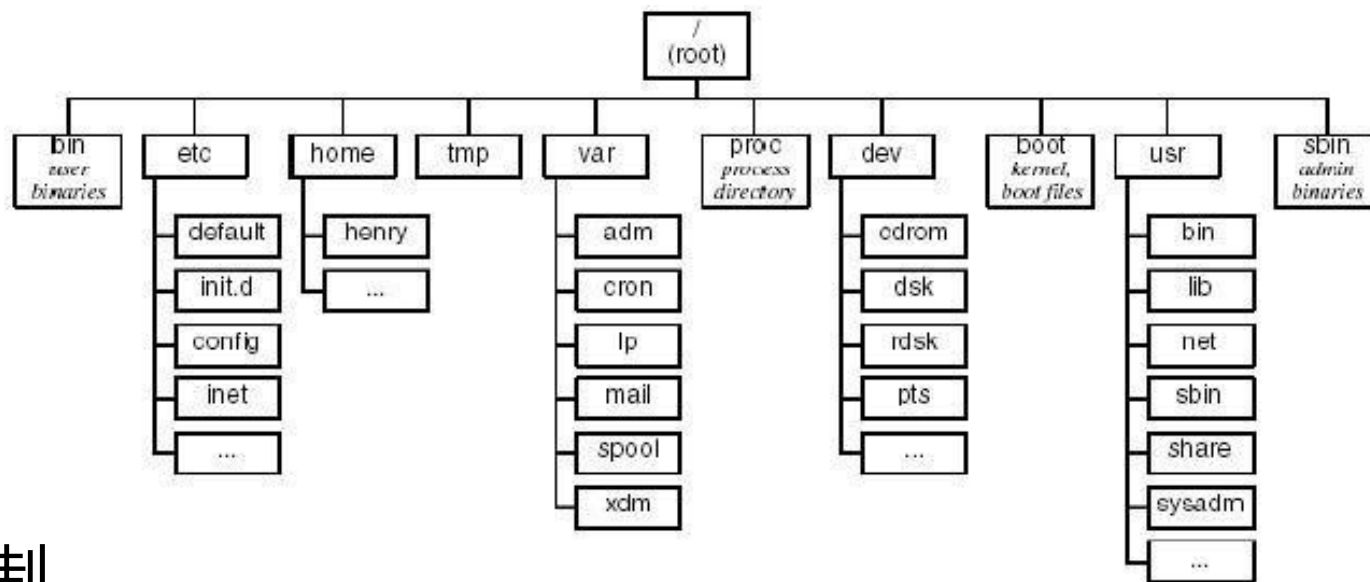
各种设备速度比较表(b/s)

04. 操作系统：（4）文件管理

- 在大多数应用中,文件是一个核心成分,除了实时应用和一些特殊应用外,应用程序的输入都是通过文件来实现的。

- 文件管理的基本功能:

- 文件存储空间的管理
- 文件操作的一般管理
- 目录管理
- 文件的读写管理和存取控制



Linux文件系统目录

04. 操作系统：（5）用户接口

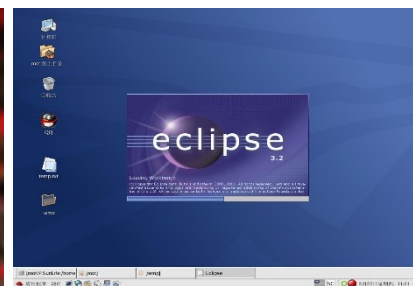
- 操作系统为用户操作提供良好的硬件接口
- 基本接口类型：
 - 命令界面(DOS, Unix, windows命令行)
 - 程序界面（系统调用界面）
 - 图形界面（windows窗口, Linux的X-window)



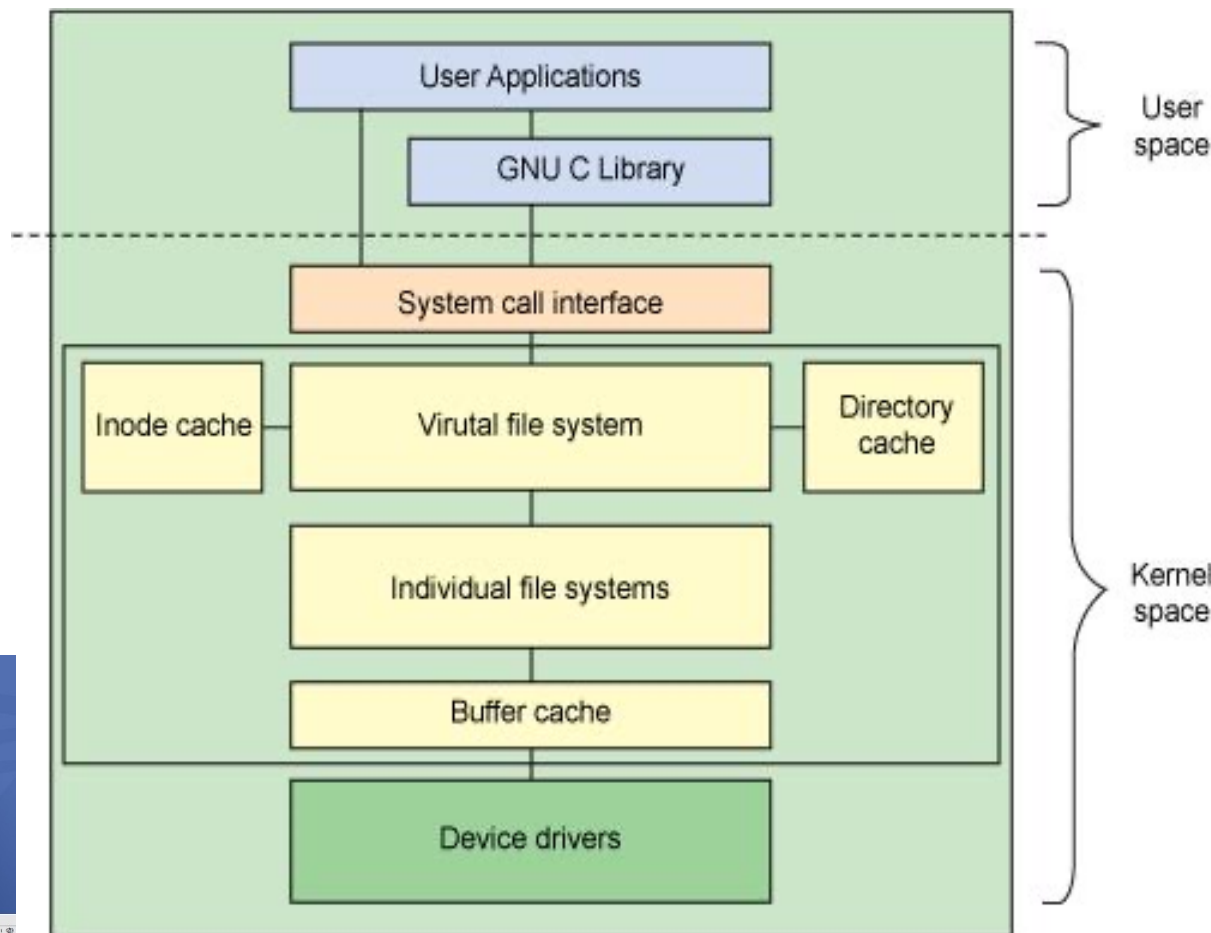
windows



macOS



Linux



04. 操作系统：附加功能和服务

■ 网络

- TCP/IP...

■ 系统工具

- shell: command line interface, ls, find/search,man
- 系统管理: ps,shutdown,mount,mkdir
- 软件开发: compilers, debuggers

■ OS 库

- I/O: data buffering and formatting
- Math: common utilities, APIs: (cos,sin,abs,sqrt)

基础知识

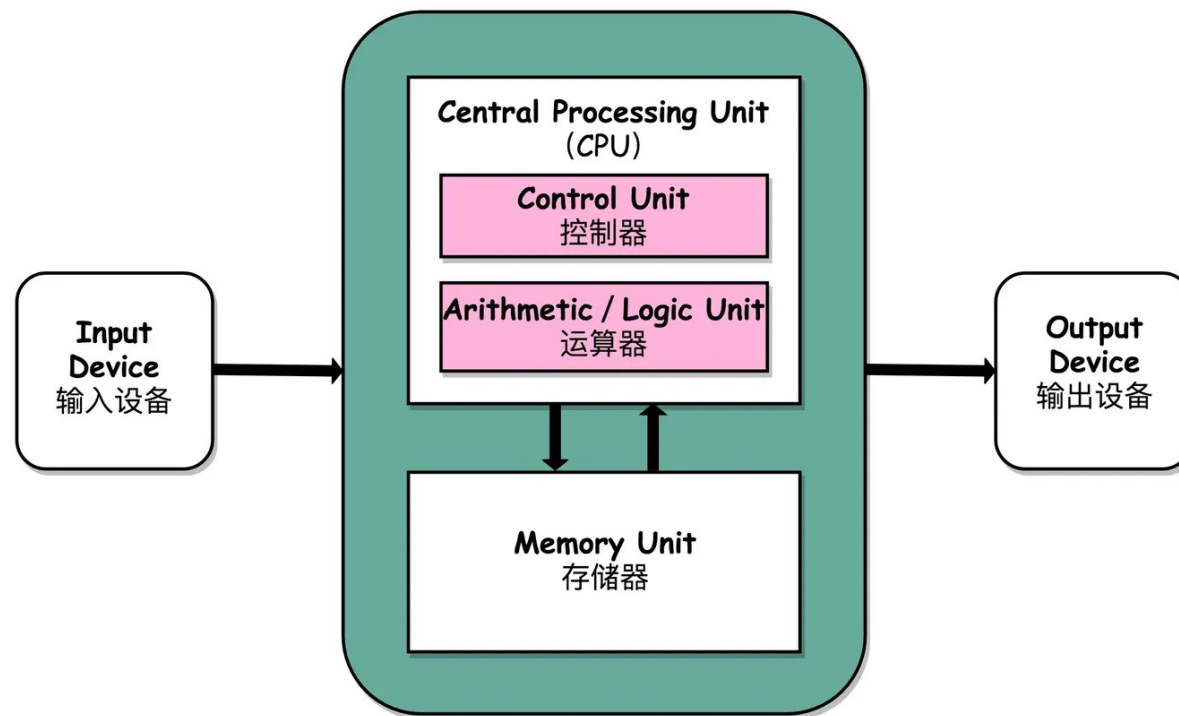
CPU指令集

计算机指令系统

单周期 vs. 多周期、流水线

存储设备

- 存储层次、存储设备
- 操作系统
- 发展、功能



谢谢！

